DS-GA 1011: Fundamentals of Natural Language Processing

Interpretability

Shauli Ravfogel

Today

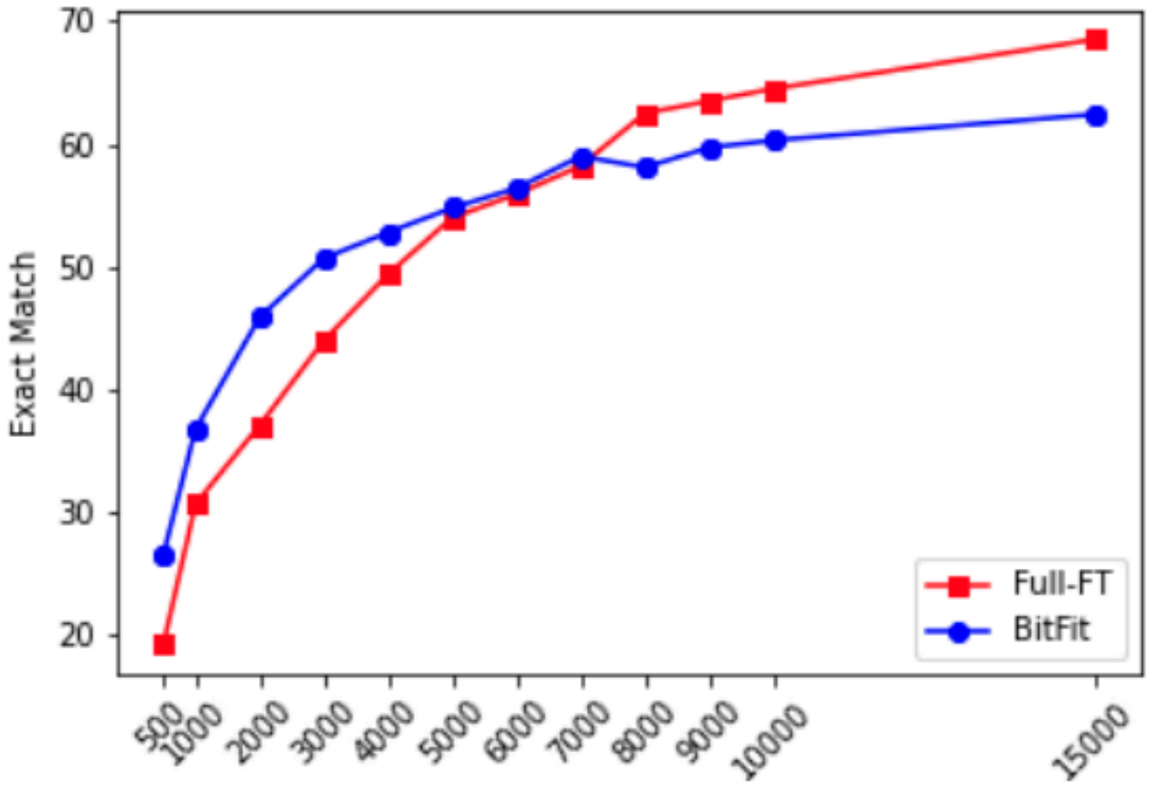
- Efficiency

- Interpretability & Explanation

Goal

- Models are scaling fast, making even fine-tuning such model resource intensive.
- **Parameter-efficient fine-tuning:** Can we update only a subset of parameters and achieve similar results as fine-tuning the full set of parameters?
- **Model Distillation:** Can we distill the ability of larger models into a smaller model?

BitFit: Only Fine-tune Bias Terms

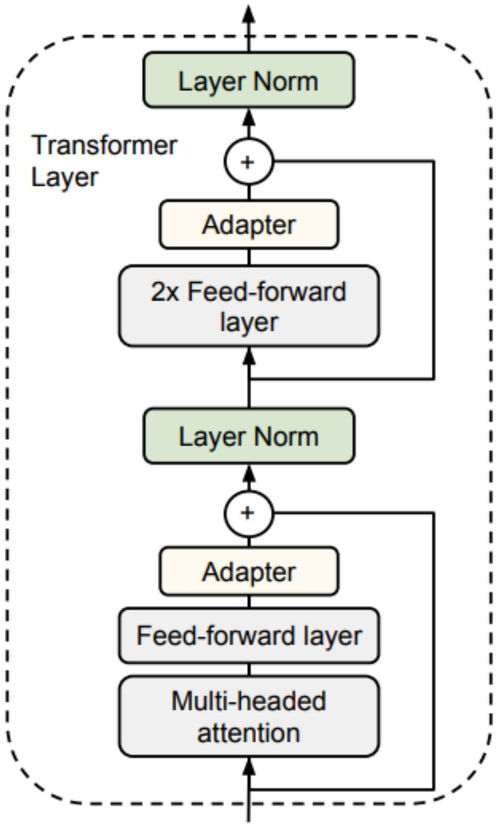


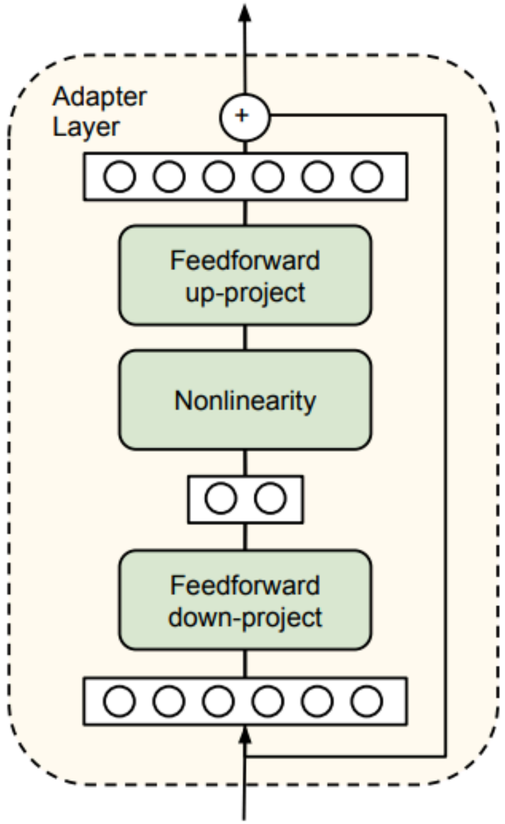
Results on GLUE benchmark, Fine-tuning BERT

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell}\mathbf{x} + \mathbf{b}_q^{m,\ell}$$
$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell}\mathbf{x} + \mathbf{b}_k^{m,\ell}$$
$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell}\mathbf{x} + \mathbf{b}_v^{m,\ell}$$

[Zaken et al, 2021]

Adaptors

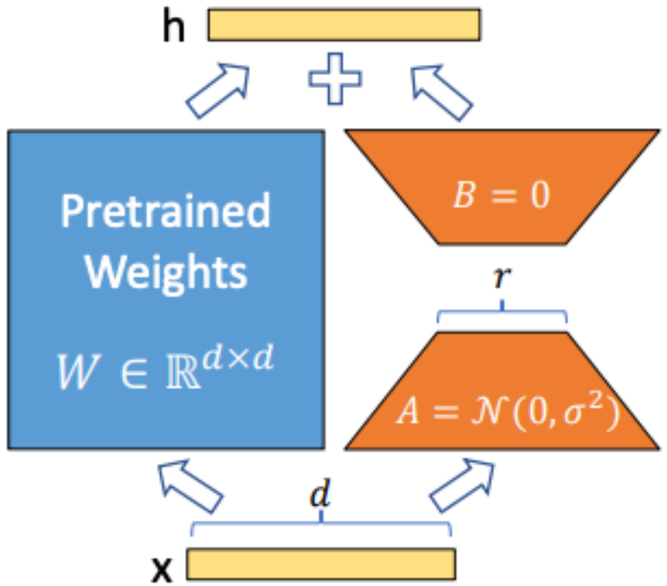


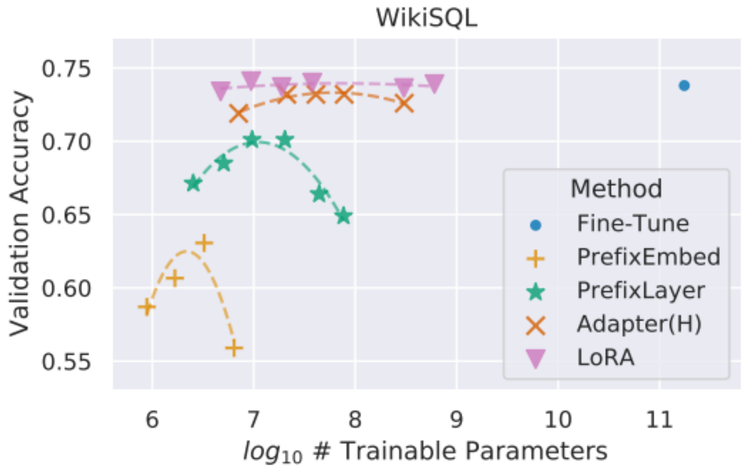


- Inject a small feedforward layer into the transformer layer and train these only

[Houlsby et al, ICML 19]

LoRA: Low-Rank Adaptation

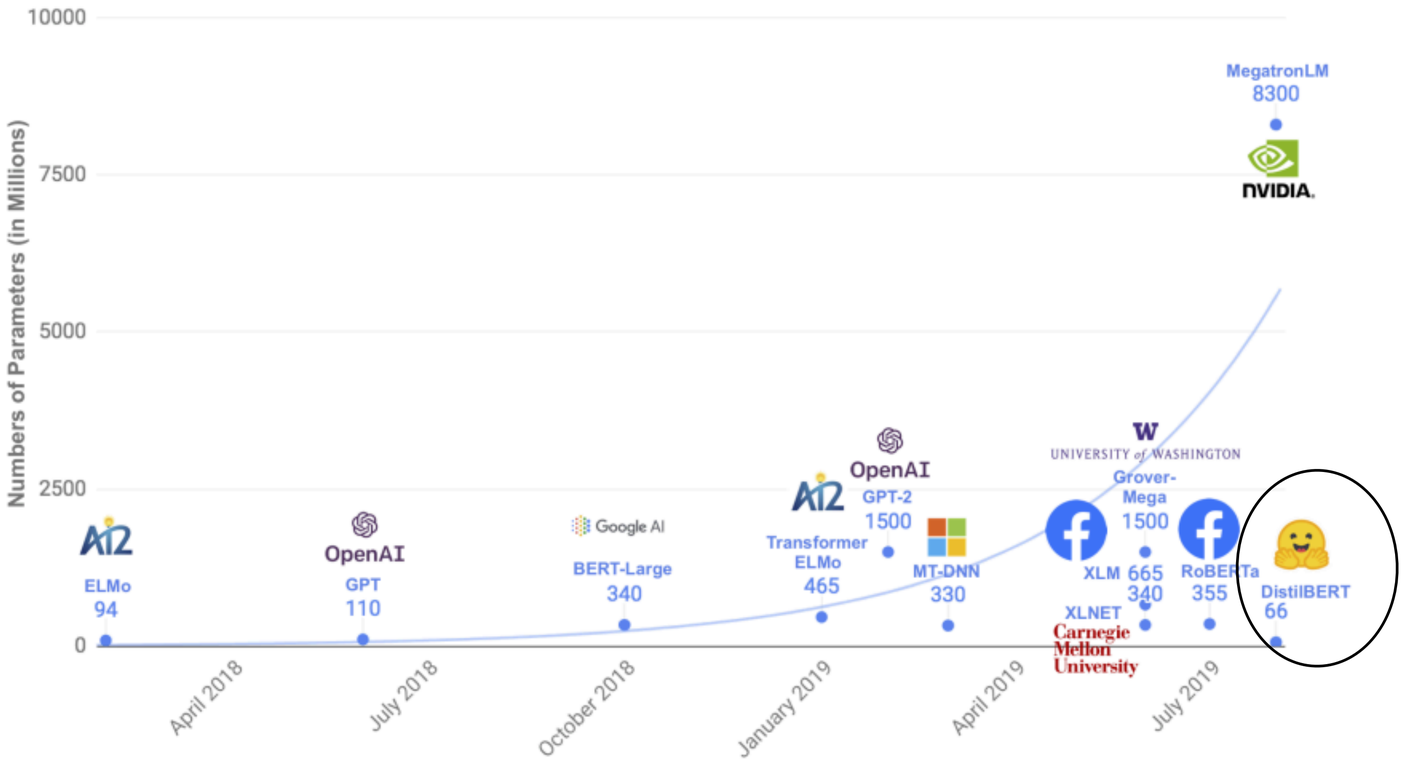




- Decomposes the weight updates into smaller matrices through low-rank decomposition
- Significantly reducing the number of trainable parameters with competitive performance

LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS
He et al,
https://arxiv.org/pdf/2106.09685

Model Distillation



- Compress large model into smaller model
- Classic formulation: Train a smaller model to imitate the **output distribution** produced by a larger model [Sanh, 2019, Turc et al, 2020]

- Symbolic Distillation: Fine-tune smaller model on large-scale data (input, output pairs) generated from larger model West et al 2022

Other Paths For Efficiency

- Quantization
  - Reducing the number of bits required to represent each parameter in a model
  - Tradeoff between performance and efficiency (latency, memory footprint)
  - Can be used with LoRA (QLoRA)

- Hardware-aware implementation
  - FlashAttention: reorders the computation of attention mechanism to make reduce I/O costs significantly (for 8K sequence, 2.7X faster)

Today

- Efficiency
- Interpretability

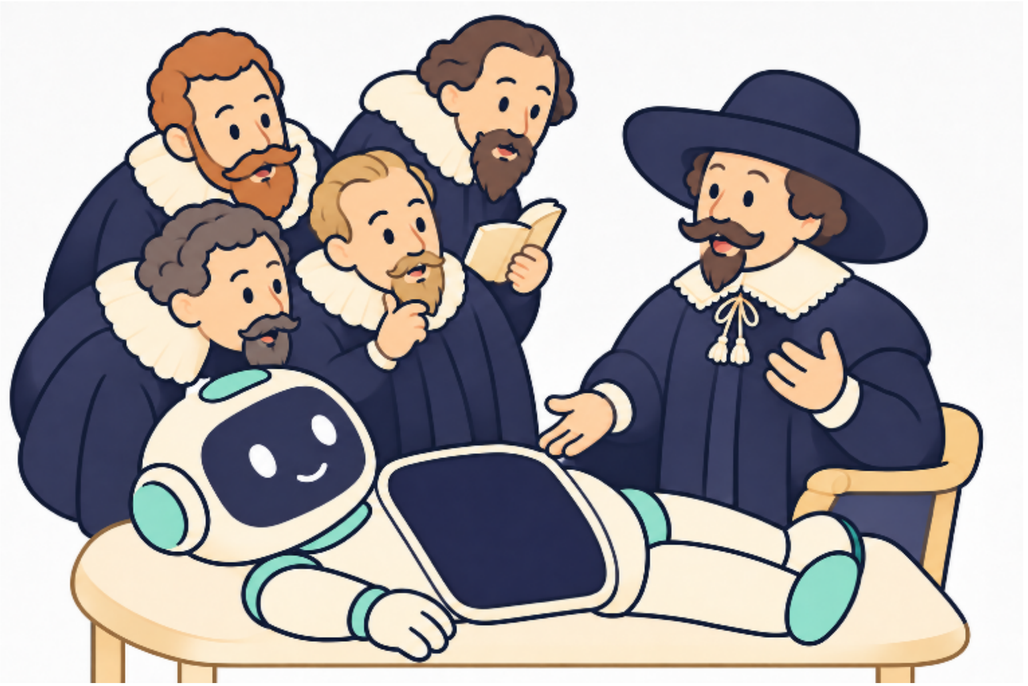
Interpretability & Explanation

- **Motivation**

- Probing

- Explaining a model decision
  - Input perturbation
  - Gradient-based methods
  - Evaluating explanation

- Other paths to interpretability



Interpretability & Explanation

- **Motivation**
- Probing
- Explaining a model decision
  - Input perturbation
  - Gradient-based methods
  - Evaluating explanation
- Other paths to interpretability

Interpretability

- **Scientific understanding** of the model
  - What concepts does it represent?
  - How does it respond to the user? Why does it choose that response?
  - How does it generate fluent text?
  - Is the representation structured? Does the model contain interpretable “circuits” (sub-components)?

LLMs are not Humans

- There is a **lot** of hype and bad science in the interpretability world (particularly on social media). Don't buy everything you read.
- Much discourse of LLM "human-like" representations, speculations about subjective experience, etc. This is mostly bad science and bad philosophy, and much of it is PR for big companies.
- What **is** a good mental model for thinking about LLMs?
- One mental model is that of a "role-player"

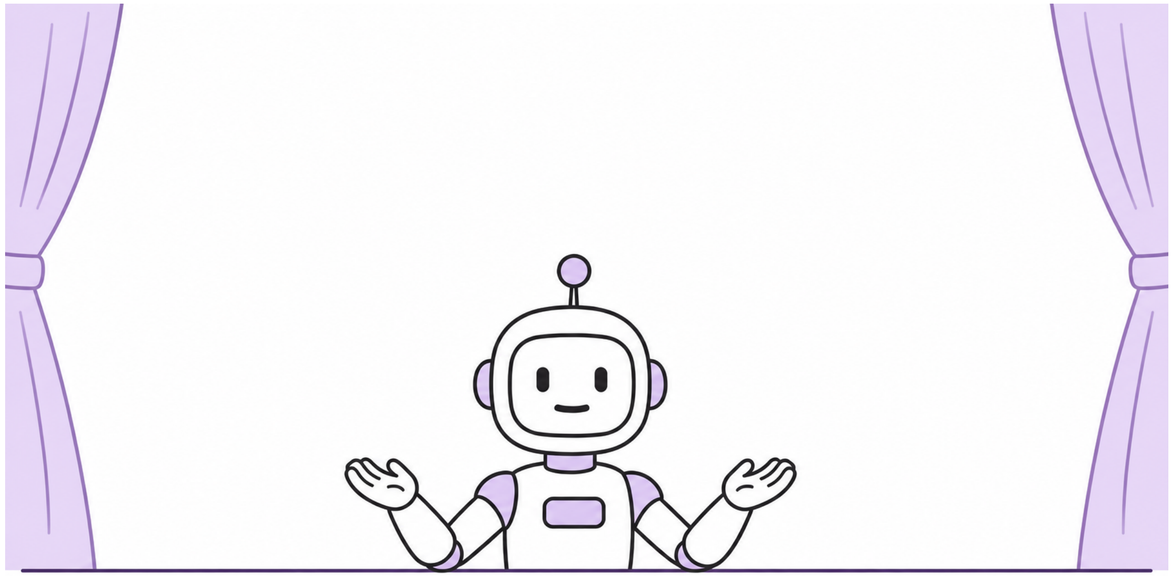
also worth reading:

(LessWrong is a sketchy venue, but this is good)

LLMs are role players

Based on slides from Yoav Goldberg 2026 LLM class

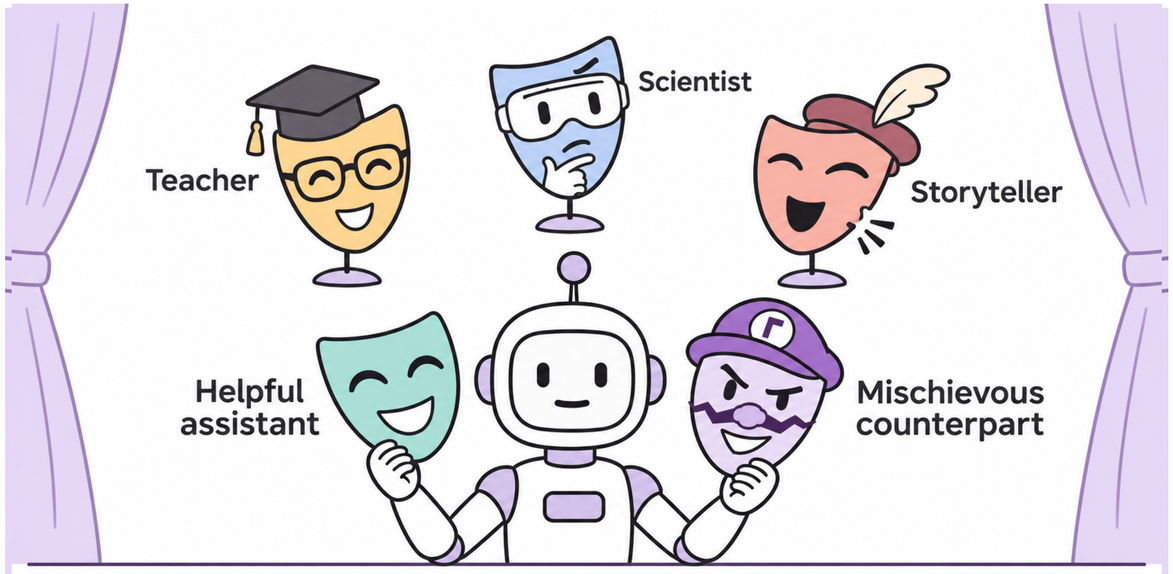
- Imagine a smart and talented actor, with much general knowledge and familarity with human behavior
- The actor is prompted iwth a situation
- The actor then assumes the role inferred by the situation and acts a single turn in that role.



LLMs are role players

Based on slides from Yoav Goldberg 2026 LLM class

- Imagine a smart and talented actor, with much general knowledge and familarity with human behavior
- The actor is prompted iwth a situation
- The actor then assumes the role inferred by the situation and acts a single turn in that role.



One model, many roles

Implications

Based on slides from Yoav Goldberg 2026 LLM class

- Much of the ability (learning about different possible roles) comes from pretraining
- Post trainign and alignment can push the model towards a particular role, the “helpful assistant”
- Roles are useful for predicting the next tokens and reducing LLM loss. There’s no reason to assume subjective experience, “intent” or other magical abilities.
- No “true self” for the model
- We can get better answers by encouraging the model to adapt a given persona

Example

Based on slides from Yoav Goldberg 2026 LLM class

- LLMs may answer many questions incorrectly, based on simulation/ role:
  - Q: What is your favorite movie?
  - A: I like many movies, it is hard to choose just one. But I really enjoyed Pulp Fiction.

Example

Based on slides from Yoav Goldberg 2026 LLM class

- LLMs may answer many questions incorrectly, based on *human misconceptions:*
  - Q: what happens when you break a mirror?
  - A: seven years of bad luck

Example

Based on slides from Yoav Goldberg 2026 LLM class

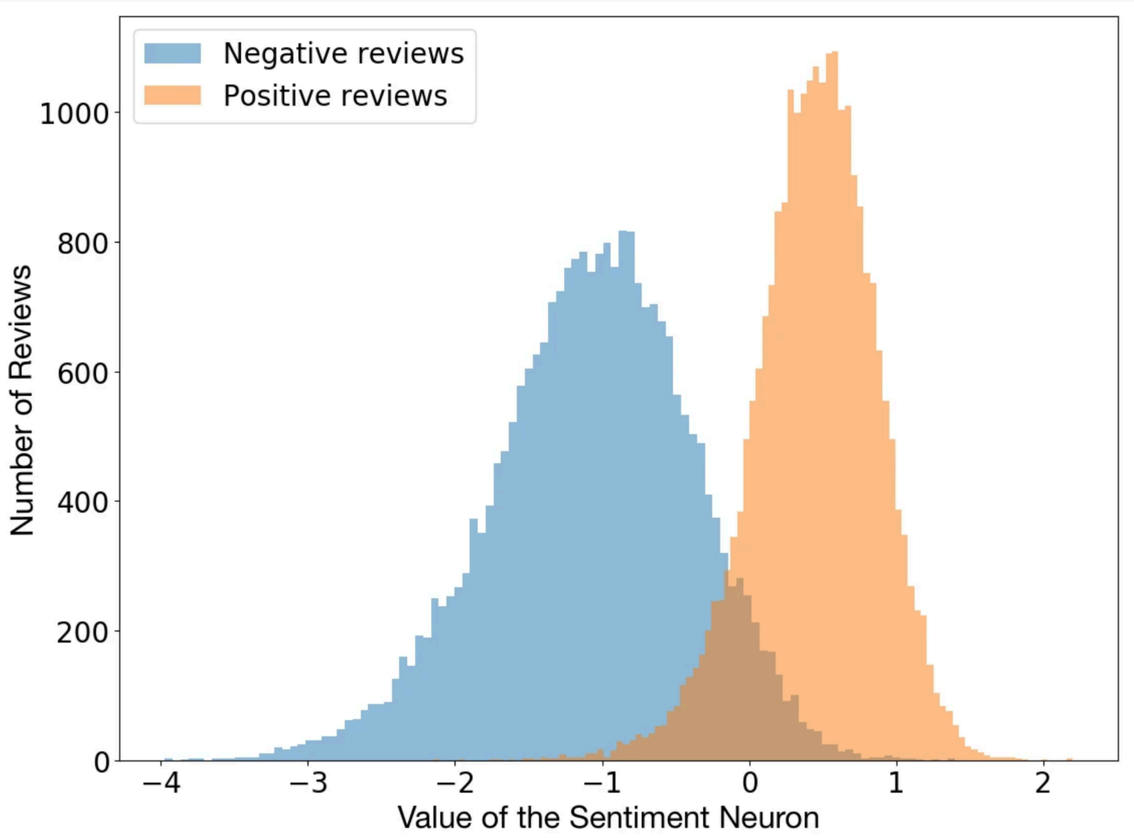
- We can fix the mistake by role-playing a different role.
- “Alice is a smart young scientist, with an IQ of 150. She was asked what happens when you break a mirror. What did she answer?”

Thinking in Training data

Based on slides from Yoav Goldberg 2026 LLM class

- Thinking in terms of fiction tropes (and data tropes) is useful:
- Why would an LLM tell you it wants to take over the world?
- How many AIs in science fiction are good and benevolent?

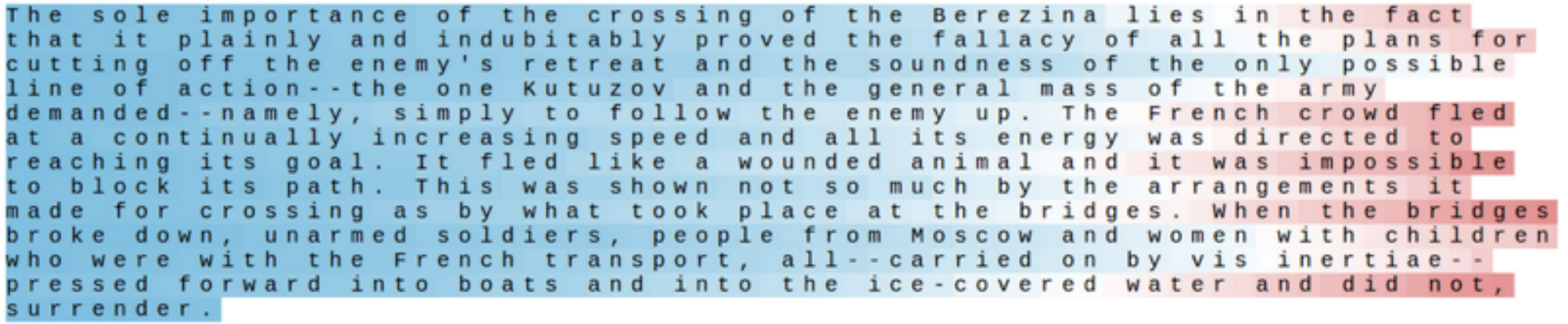
First Attempt: Neuron Interpretability



https://openai.com/index/unsupervised-sentiment-neuron/

First Attempt: Neuron Interpretability

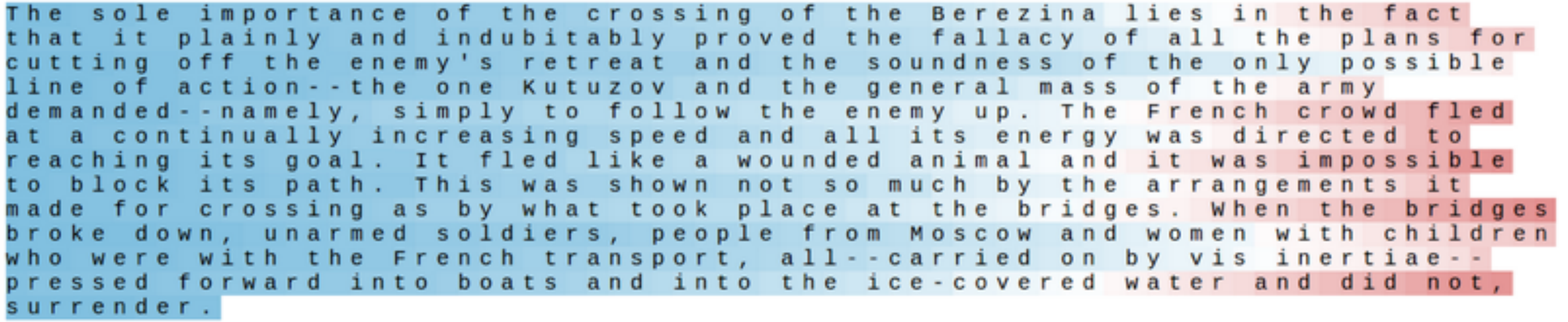
Cell sensitive to position in line:



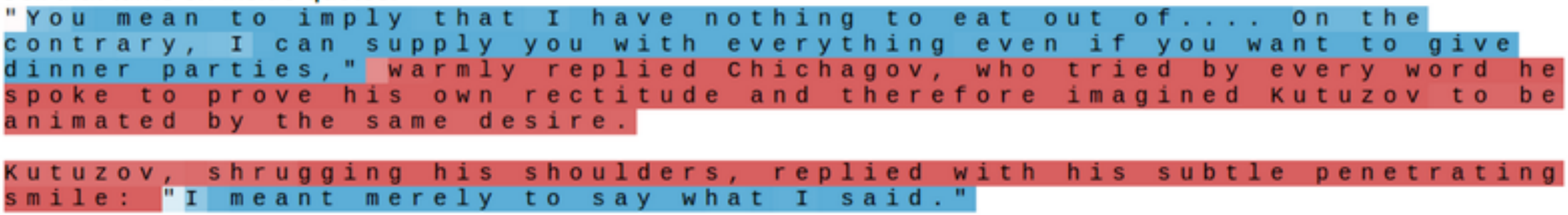
Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks

First Attempt: Neuron Interpretability

Cell sensitive to position in line:



Cell that turns on inside quotes:



Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks

First Attempt: Neuron Interpretability

Cell that is sensitive to the depth of an expression:

```
#ifdef CONFIG_AUDITSYSCALL
static inline int audit_match_class_bits(int class, u32 *mask)
{
  int i;
  if (classes[class]) {
    for (i = 0; i < AUDIT_BITMASK_SIZE; i++)
      if (mask[i] & classes[class][i])
        return 0;
  }
  return 1;
}
```

Cell that turns on inside quotes:

Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks

First Attempt: Neuron Interpretability

Cell that is sensitive to the depth of an expression:

```
#ifdef CONFIG_AUDITSYSCALL
static inline int audit_match_class_bits(int class, u32 *mask)
{
  int i;
  if (classes[class]) {
    for (i = 0; i < AUDIT_BITMASK_SIZE; i++)
      if (mask[i] & classes[class][i])
        return 0;
```

Cell that might be helpful in predicting a new line. Note that it only turns on for some ")":

```
char *audit_unpack_string(void **bufp, size_t *remain, si
{
  char *str;
  if (!*bufp || (len == 0) || (len > *remain))
    return ERR_PTR(-EINVAL);
  /* Of the currently implemented string fields, PATH_MAX
   * defines the longest valid length.
   */
  if (len > PATH_MAX)
    return ERR_PTR(-ENAMETOOLONG);
  str = kmalloc(len + 1, GFP_KERNEL);
  if (unlikely(!str))
    return ERR_PTR(-ENOMEM);
  memcpy(str, *bufp, len);
  str[len] = 0;
  *bufp += len;
  *remain -= len;
  return str;
}
```

Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks

First Attempt: Neuron Interpretability

Cell that is sensitive to the depth of an expression:

```
#ifdef CONFIG_AUDITSYSCALL
static inline int audit_match_class_bits(int class, u32 *mask)
{
  int i;
  if (classes[class]) {
    for (i = 0; i < AUDIT_BITMASK_SIZE; i++)
      if (mask[i] & classes[class][i])
        return 0;
```

Cell that might be helpful in predicting a new line. Note that it only turns on for some ")":

```
char *audit_unpack_string(void **bufp, size_t *remain, si
{
  char *str;
  if (!*bufp || (len == 0) || (len > *remain))
    return ERR_PTR(-EINVAL);
  /* Of the currently implemented string fields, PATH_MAX
   * defines the longest valid length.
   */
  if (len > PATH_MAX)
    return ERR_PTR(-ENAMETOOLONG);
  str = kmalloc(len + 1, GFP_KERNEL);
  if (unlikely(!str))
    return ERR_PTR(-ENOMEM);
  memcpy(str, *bufp, len);
  str[len] = 0;
  *bufp += len;
  *remain -= len;
  return str;
}
```

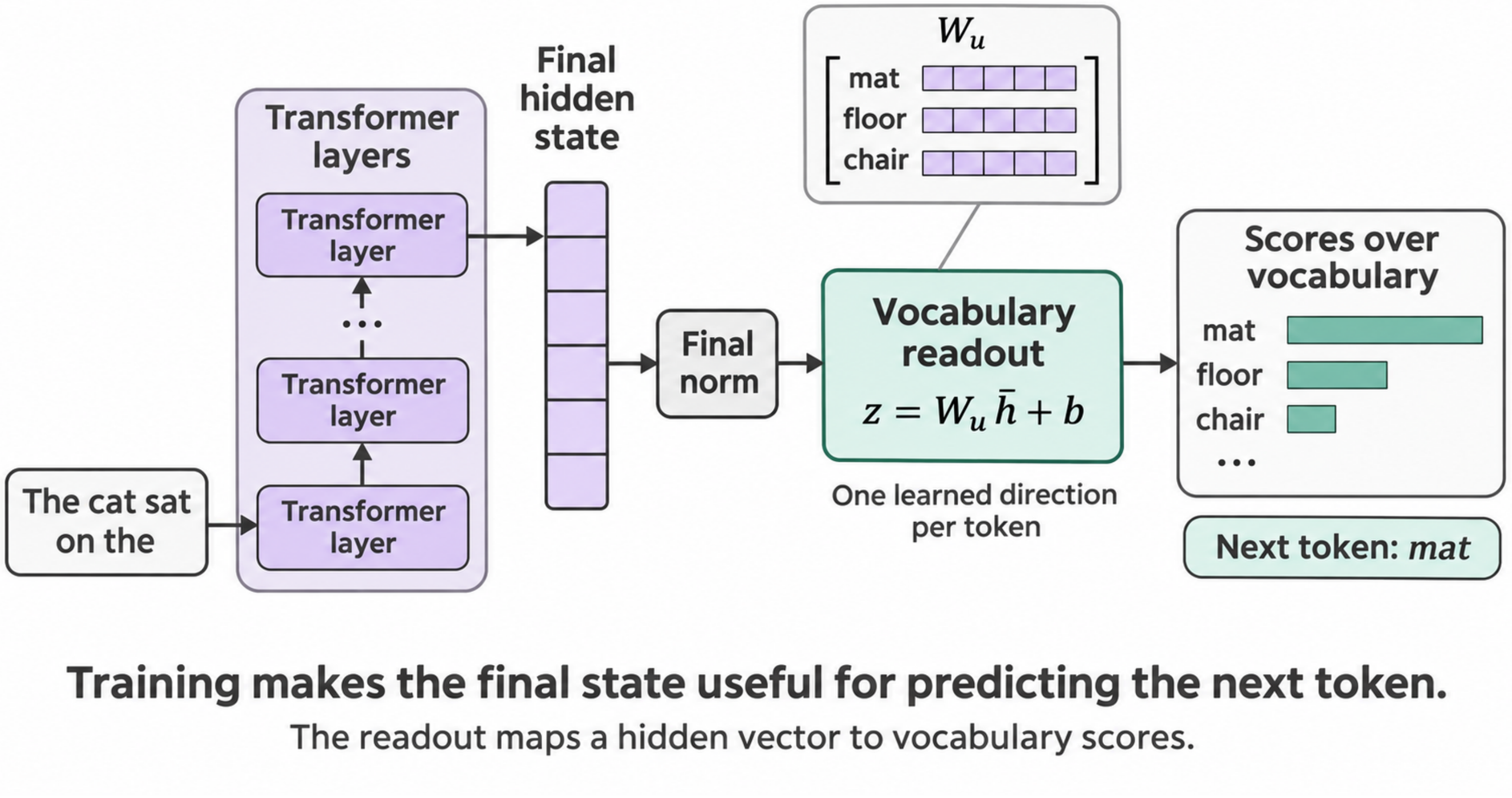
In Neuroscience: "the grandma neuron"
What's the problem with the idea?

Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks

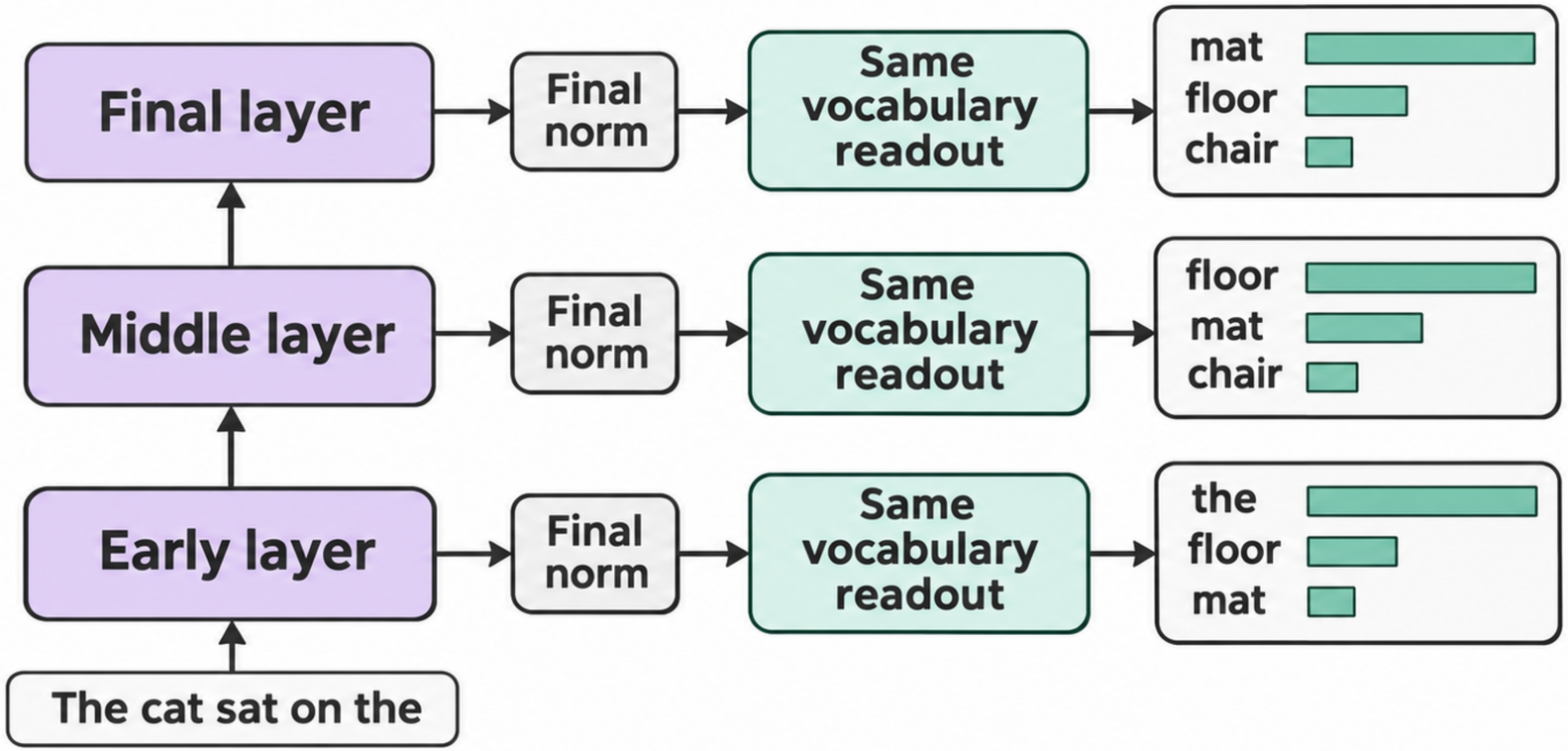
Trasnfer the Representation to another space

- Individual neurons are not necessarily interpretable.
- We can **project** them into a more interpretable space
  - Logit lens: project to the vocabulary space
  - J-lens: logit lens + a linear approximation of layers $\ell \to L$ (the averaged Jacobian)
  - SAEs: learn a nonlinear projection

Logit Lens

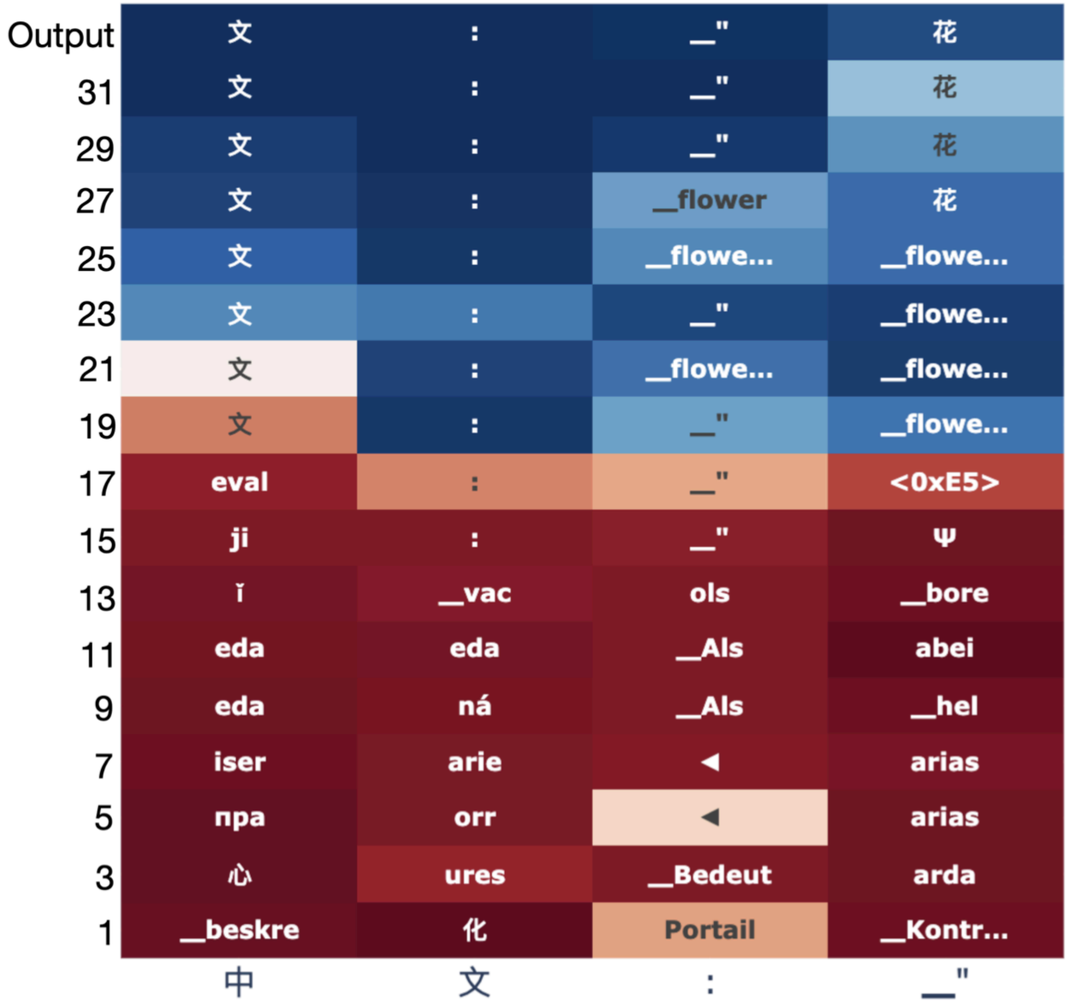

The cat sat on the
Transformer layer
Transformer layer
Transformer layer
Transformer layers
Final hidden state
Final norm
W_u
mat
floor
chair
Vocabulary readout
$z = W_u \bar{h} + b$
One learned direction per token
Scores over vocabulary
mat
floor
chair
Next token: mat
Training makes the final state useful for predicting the next token.
The readout maps a hidden vector to vocabulary scores.

Logit Lens


Final layer
Middle layer
Early layer
The cat sat on the
Final norm
Final norm
Final norm
Same vocabulary readout
Same vocabulary readout
Same vocabulary readout
mat
floor
chair
floor
mat
chair
the
floor
mat

Logit Lens

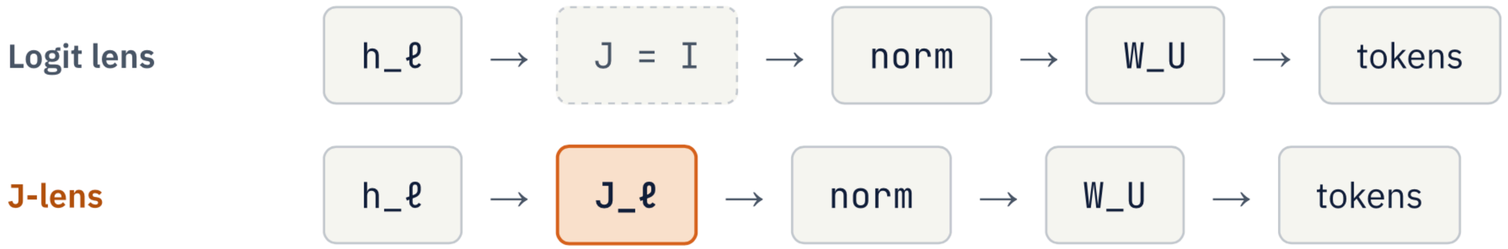
Figure 1: **Illustration of logit lens,** which applies language modeling head (here, Llama-2-7B) prematurely to latent embeddings in intermediate layers, yielding one next-token distribution per position (x-axis) and layer (y-axis). We show final tokens of translation prompt (cf. Sec. 3.3) ending with “Français: "fleur" - 中文: "”” (where “中文” means “Chinese”). Final layer correctly ranks “花” (translation of “fleur”) on top, whereas intermediate layers decode English “flower”. Color indicates entropy of next-token distributions from low (blue) to high (red). (Plotting tool: Belrose et al. (2023).)



Wendler, Chris, et al. "Do llamas work in english? on the latent language of multilingual transformers." Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024.

J-lens

- Use the model's own Jacobian to translate layer ℓ into the final layer's coordinates.



$$\text{lens}(h_\ell) = \text{softmax}(W_U \cdot \text{norm}(J_\ell \, h_\ell))$$

J_ℓ answers: *if h_ℓ moved a little in this direction, how would the final residual stream move?* Setting $J_\ell = I$ gives back the logit lens.

Anthropic, A global workspace in language models

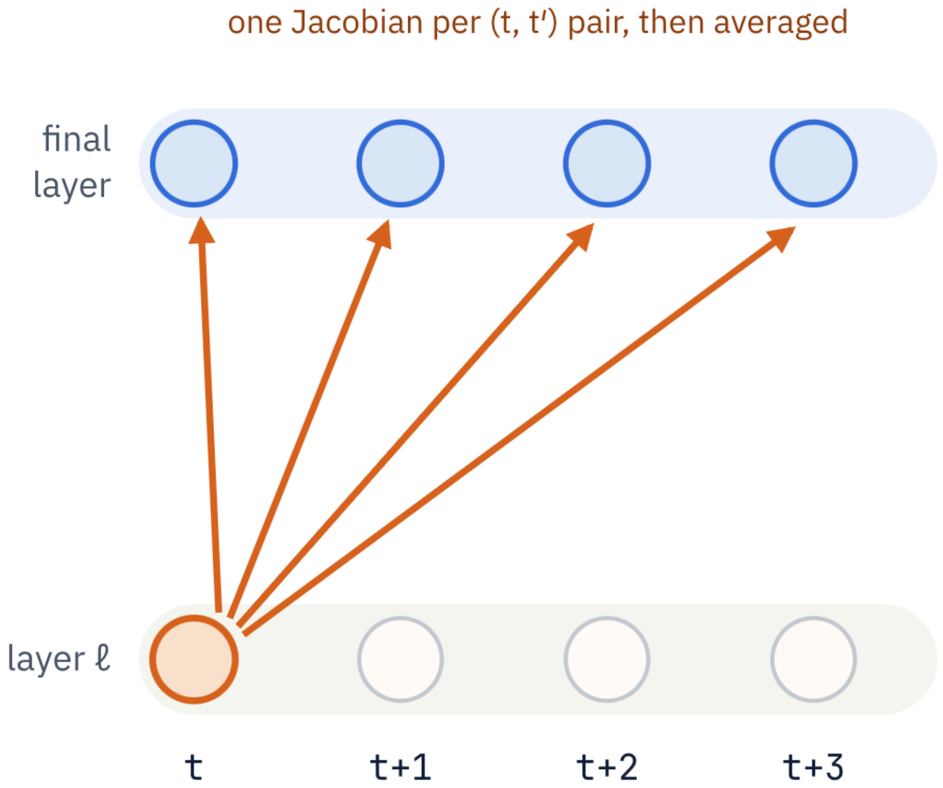
J-lens

An average Jacobian, computed once

$$J_\ell = E[\ \partial h_{final,t'} / \partial h_{\ell,t}\ ]$$

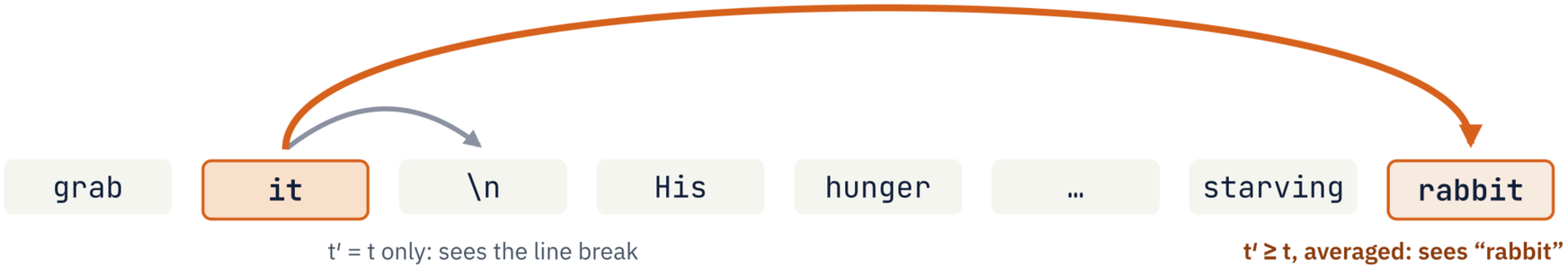
Averaged over the source position t, every later position $t' \geq t$, and about 1,000 pretraining-like prompts.

Result: one d_model × d_model matrix per layer, from backprop. No training, no labels.



Anthropic, A global workspace in language models

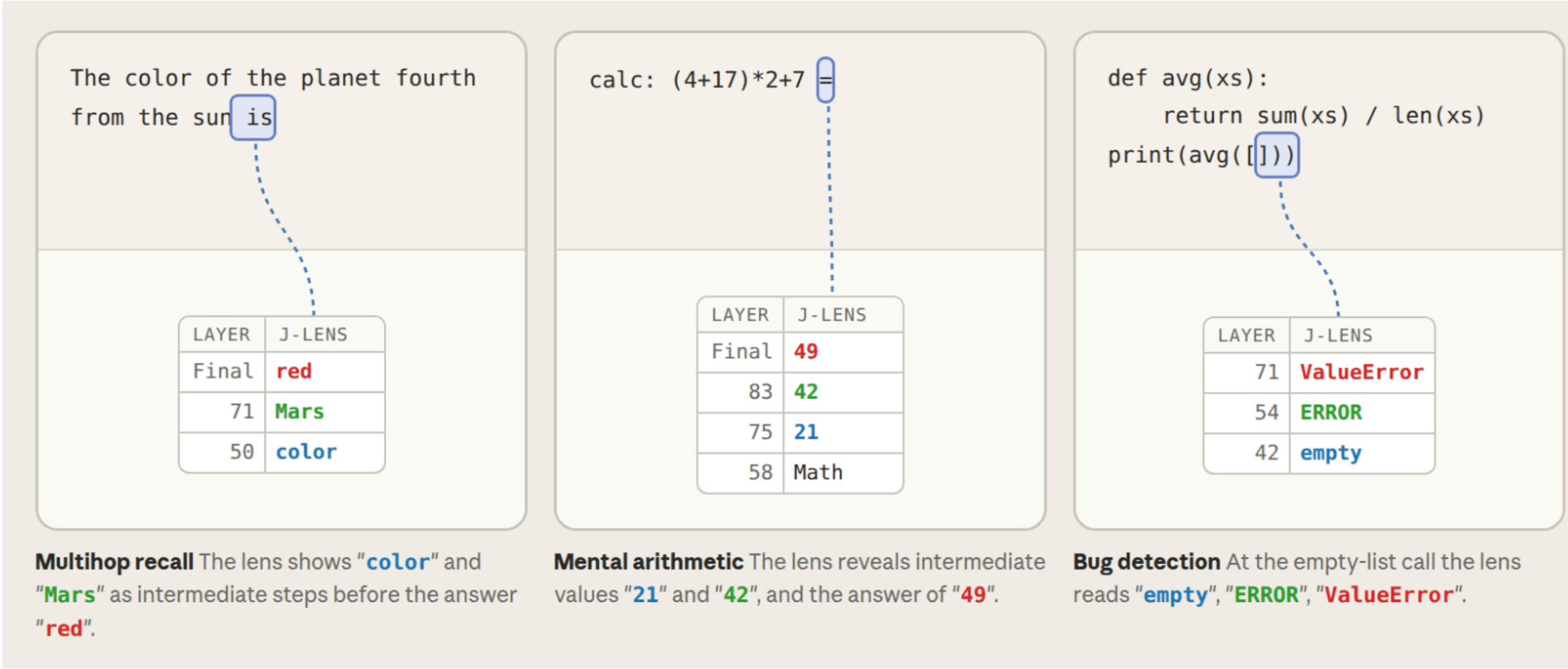
J-lens



Each row of W_U J_ℓ is a J-lens vector: the direction at layer ℓ that the model is “poised to say” as one particular token.

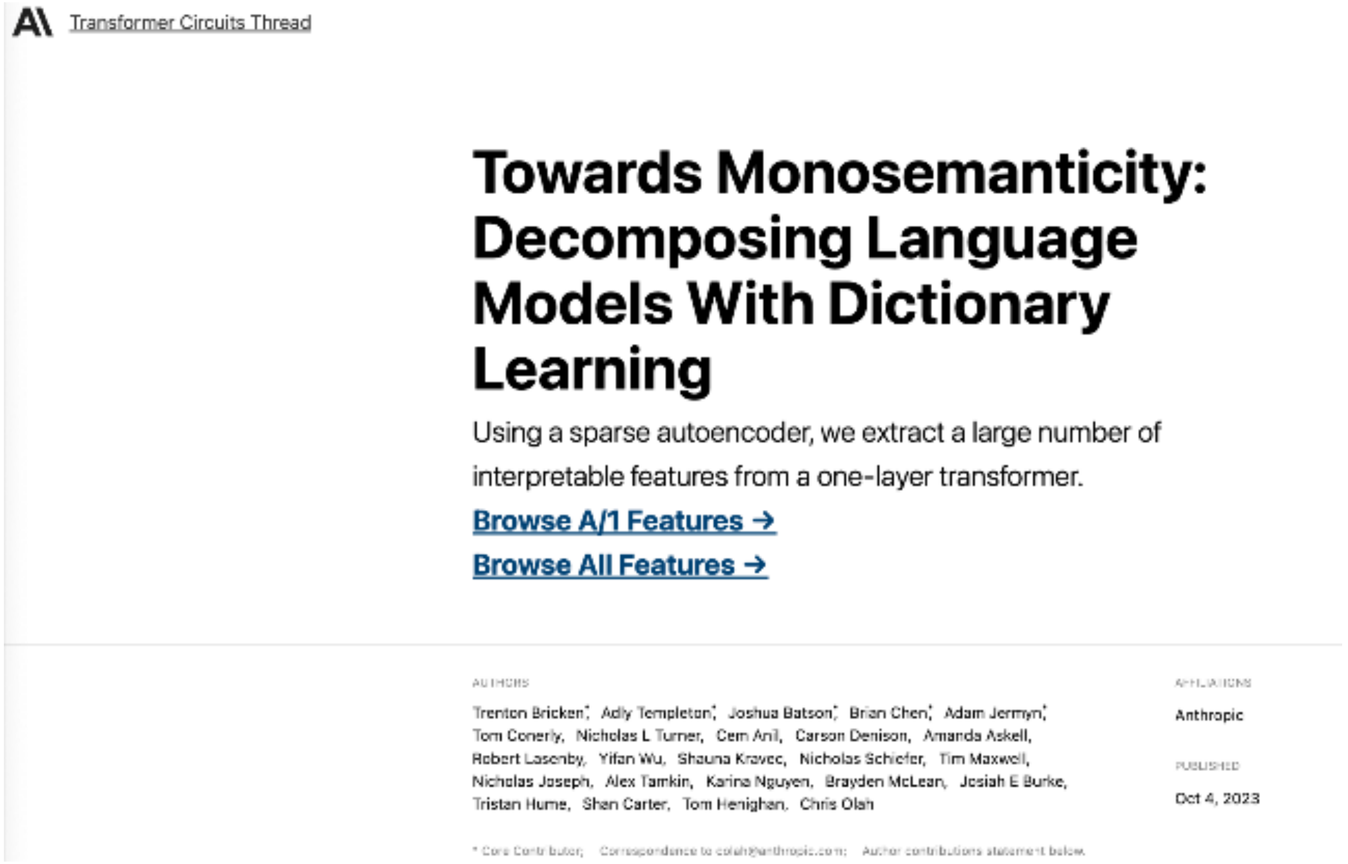
Anthropic, A global workspace in language models

J-lens



Anthropic, A global workspace in language models

Sparse Auto Encoders (SAEs)



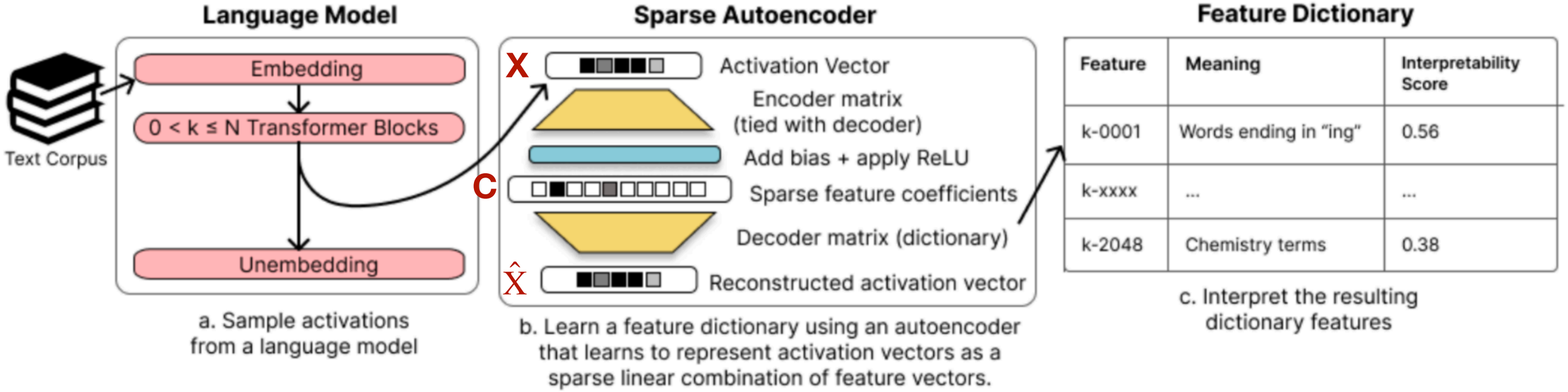
Wendler, Chris, et al. "Do llamas work in english? on the latent language of multilingual transformers." Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024.

SAEs

- Collect the activations on many, many instances
- Train an autoencoder on each layer representation with a L1 regularization or variants (to encourage sparsity)
- Hope the resulting representation will be more interpretable

Wendler, Chris, et al. "Do llamas work in english? on the latent language of multilingual transformers." Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024.

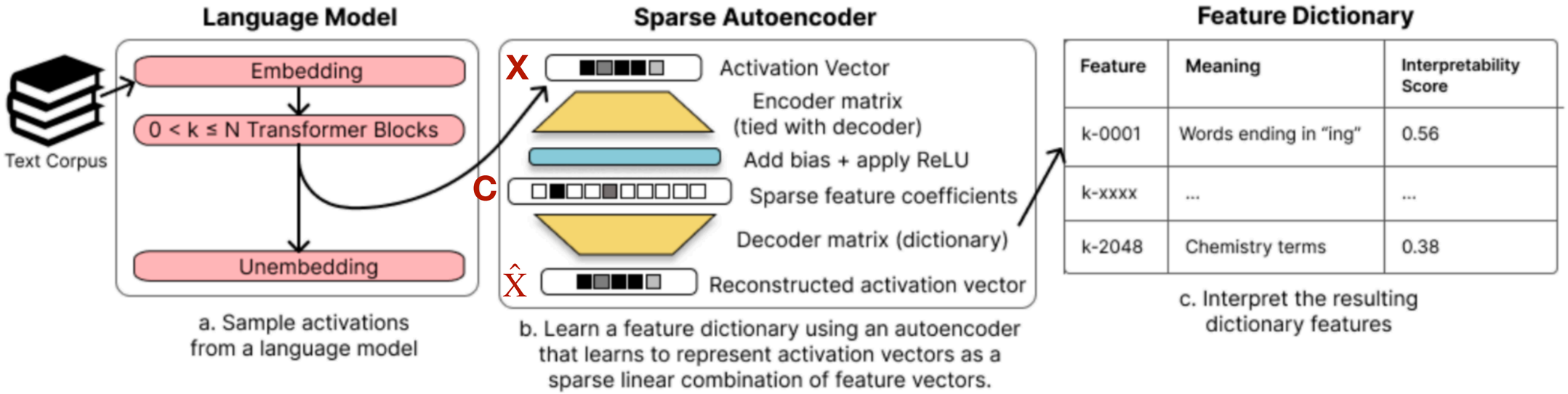
SAEs



(ICLR 2024) Sparse Autoencoders Find Highly Interpretable Features in Language Models Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, Lee Sharkey

$$\mathcal{L}(\mathbf{x}) = \underbrace{||\mathbf{x} - \hat{\mathbf{x}}||_2^2}_{\text{Reconstruction loss}} + \underbrace{\alpha||\mathbf{c}||_1}_{\text{Sparsity loss}}$$

SAEs

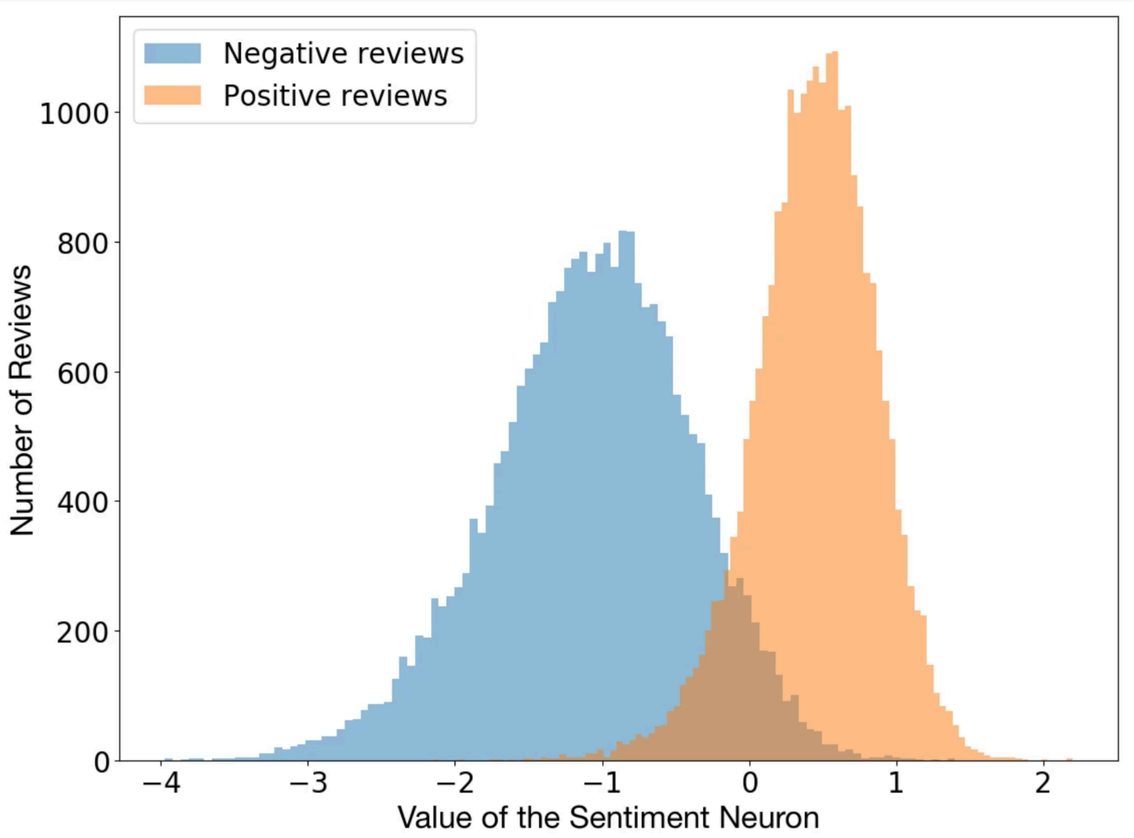


(ICLR 2024) Sparse Autoencoders Find Highly Interpretable Features in Language Models Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, Lee Sharkey

$$\mathcal{L}(\mathbf{x}) = \underbrace{||\mathbf{x} - \hat{\mathbf{x}}||_2^2}_{\text{Reconstruction loss}} + \underbrace{\alpha||\mathbf{c}||_1}_{\text{Sparsity loss}}$$

To interpret the features: ask humans's/LLMs what's special about data that activates a given feature

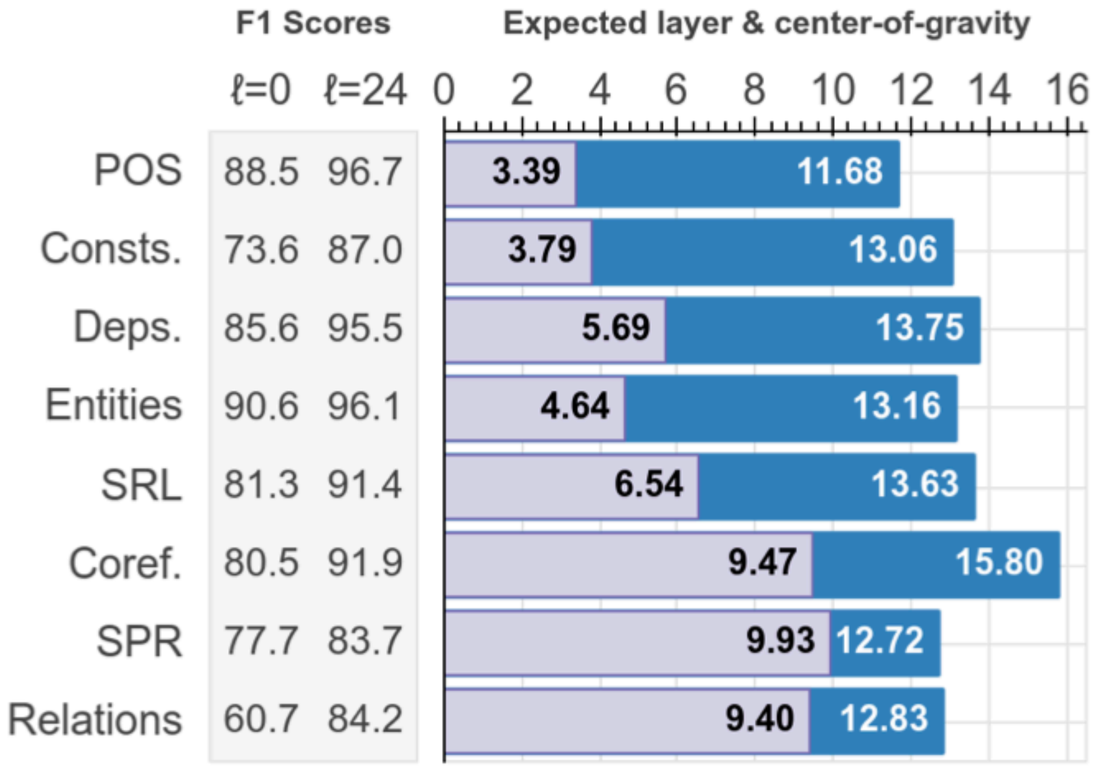
Probing



https://openai.com/index/unsupervised-sentiment-neuron/

- Key idea:
  Given an pretrained LM (e.g., BERT, LlaMa) and an input text, we use its output representation as an input to train a classifier to predict a property of the input text
- If successful, means that property is encoded in the model representation.
- If not, ?

Probing Linguistic Tasks



- Expected layer: layer at which model correctly labels an example
- Center-of-gravity: the layer which captures the information needed to correctly labels an example
- Different layers capture different linguistic information!

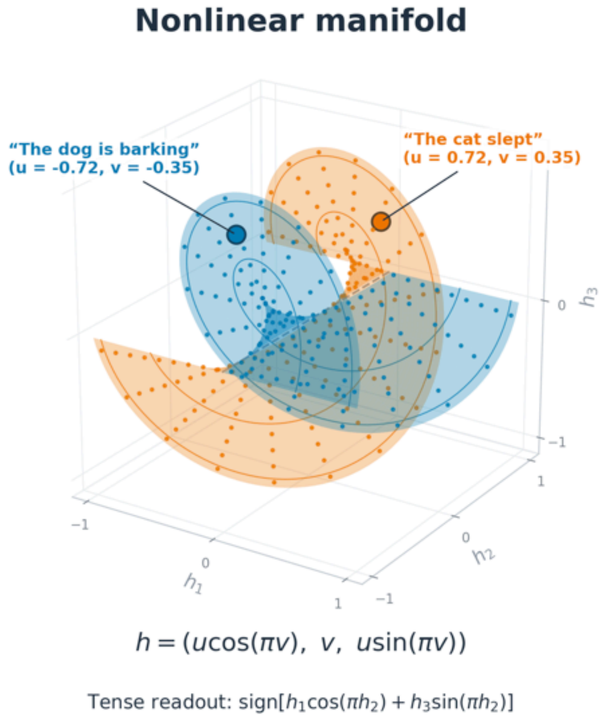
BERT Rediscovers the Classical NLP Pipeline
ACL 19, Tenney et al, https://aclanthology.org/P19-1452.pdf

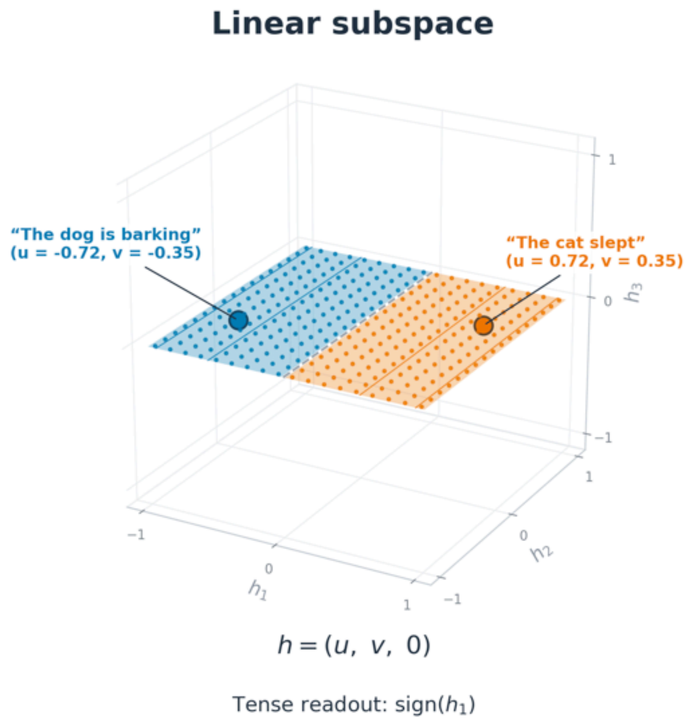
Spurious Signal

- Suppose that the encoding of the text H(T) doesn't lose information on the sentence
- Then, information-theoretically, **any** computable property of the original sentence can be decoded from the representation
- We need **computationally constrained** probes (e.g., linear).

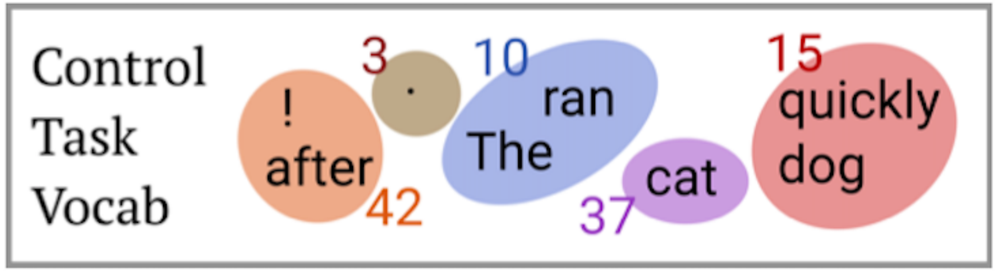
Spurious Signal

- Consider sentences like “The cat slept” or “the dog is barking”
- Let u be a tense coordinate (positive = past, negative = present)
- Let v be a toy content coordinate (positive = cats, negative = dogs)



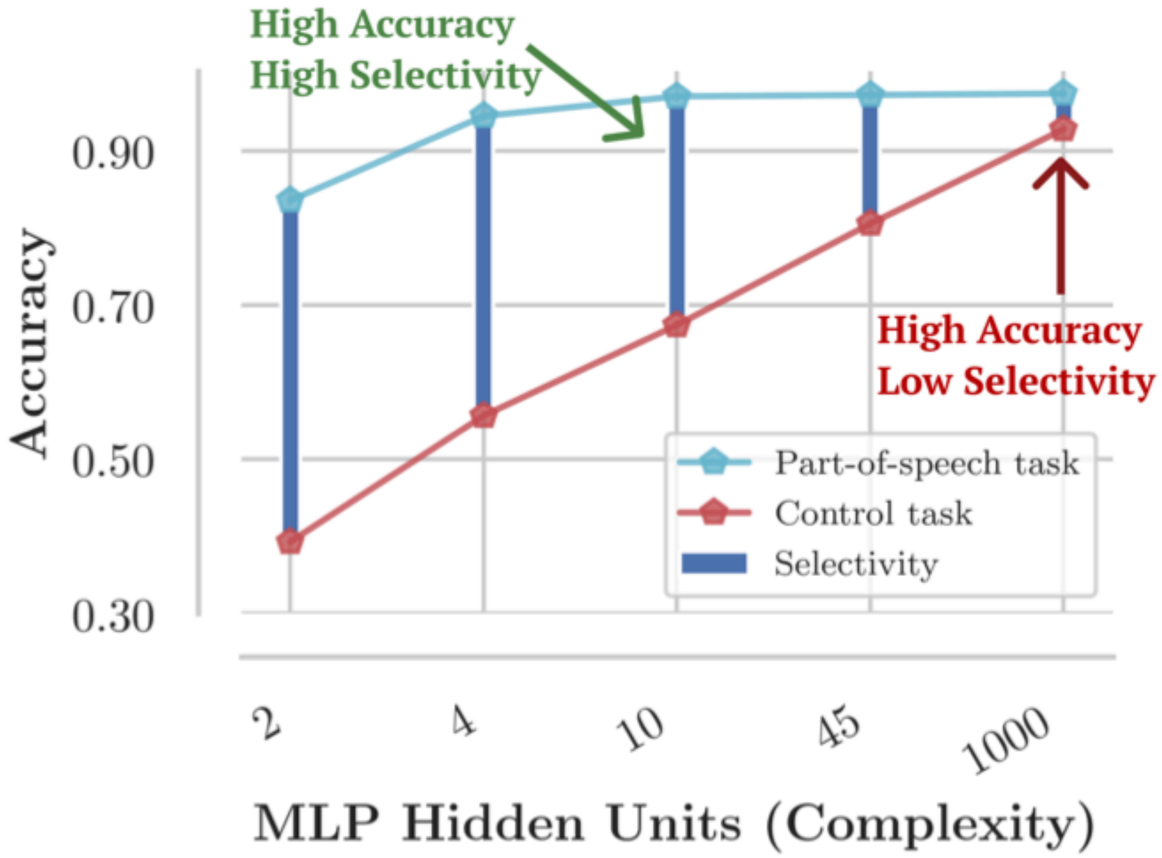


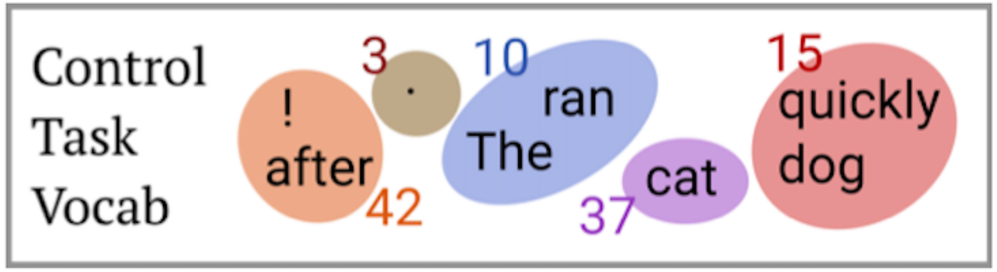
Spurious Signal



Hewitt, John, and Percy Liang. "Designing and interpreting probes with control tasks." *Proceedings of the 2019 conference on empirical methods in natural language processing*.

Spurious Signal



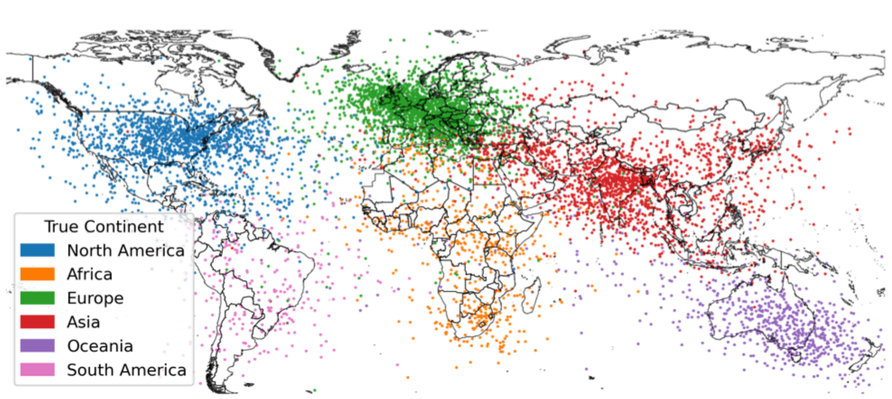


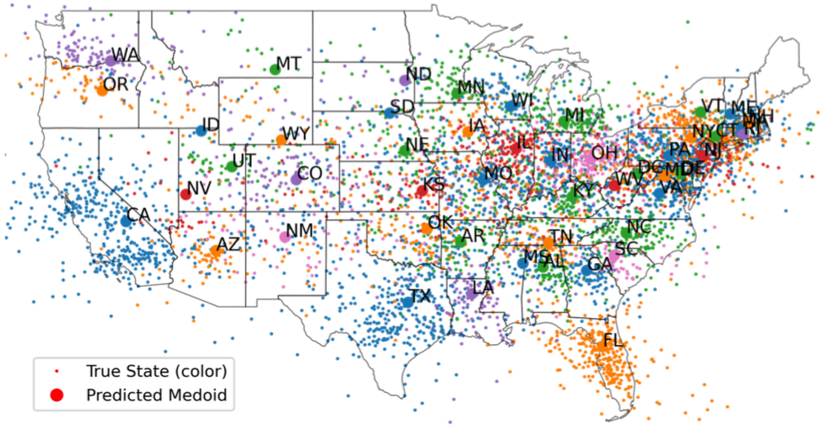
Sentence 1	The	cat	ran	quickly	.
Part-of-speech	DT	NN	VBD	RB	.
Control task	10	37	10	15	3
Sentence 2	The	dog	ran	after	!
Part-of-speech	DT	NN	VBD	IN	.
Control task	10	15	10	42	42

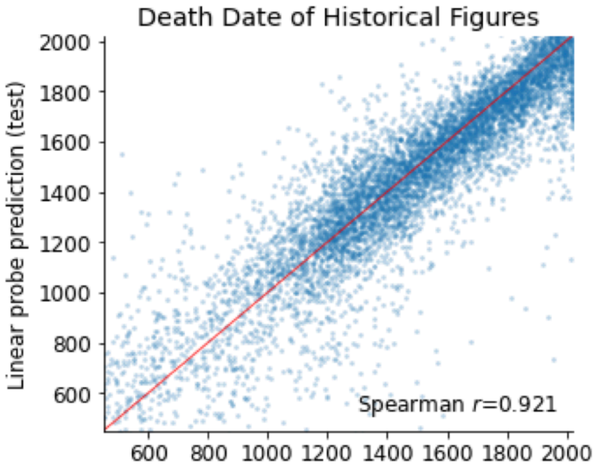
Hewitt, John, and Percy Liang. "Designing and interpreting probes with control tasks." *Proceedings of the 2019 conference on empirical methods in natural language processing*.

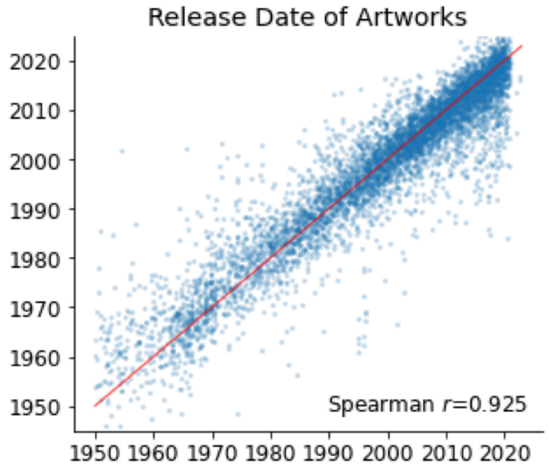
LM Activations Encode “Space” and “Time”

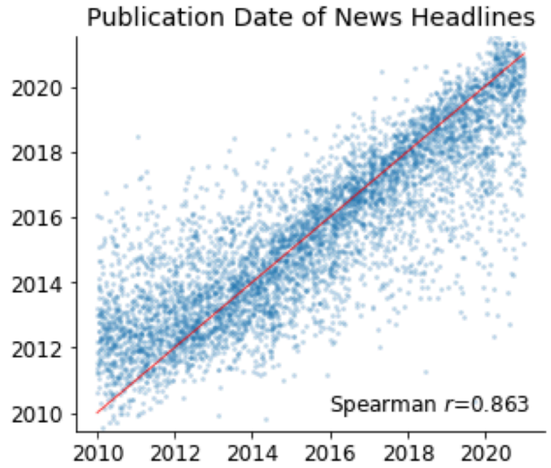
- We can also probe physical attributes of the world.







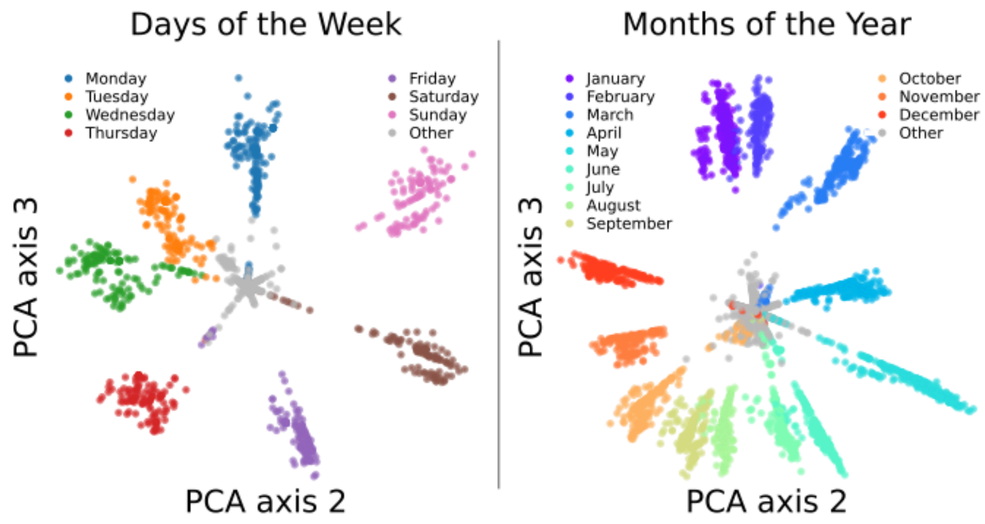


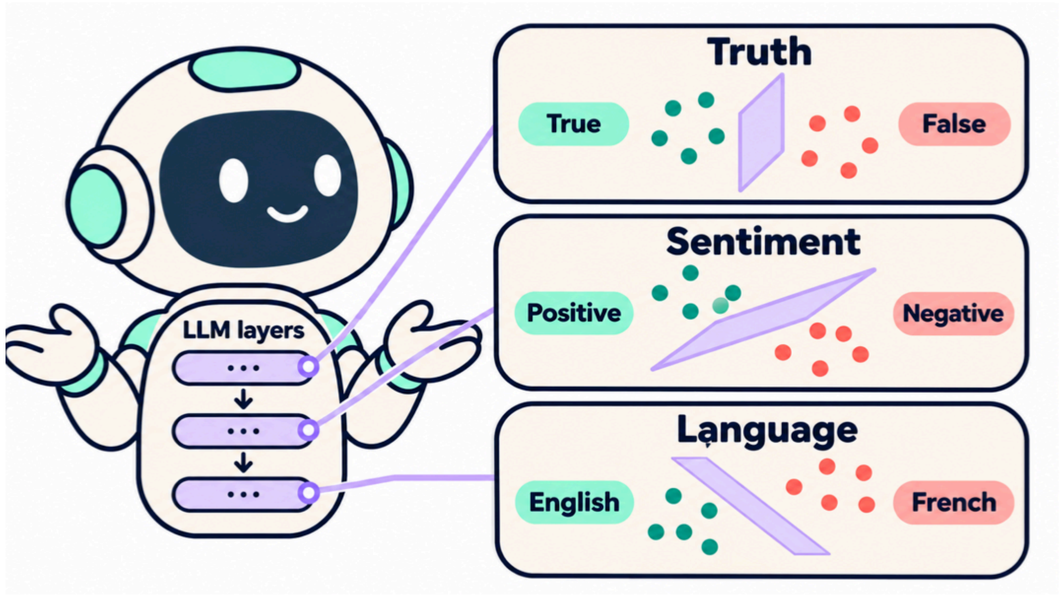


Language Models Represent Space and Time
Wes Gurnee, Max Tegmark
ICLR 2024 https://arxiv.org/abs/2310.02207

The Linear Representation Hypothesis

- Empirically, much information in LLMs is **linearly encoded**.
- However, this is not always true.



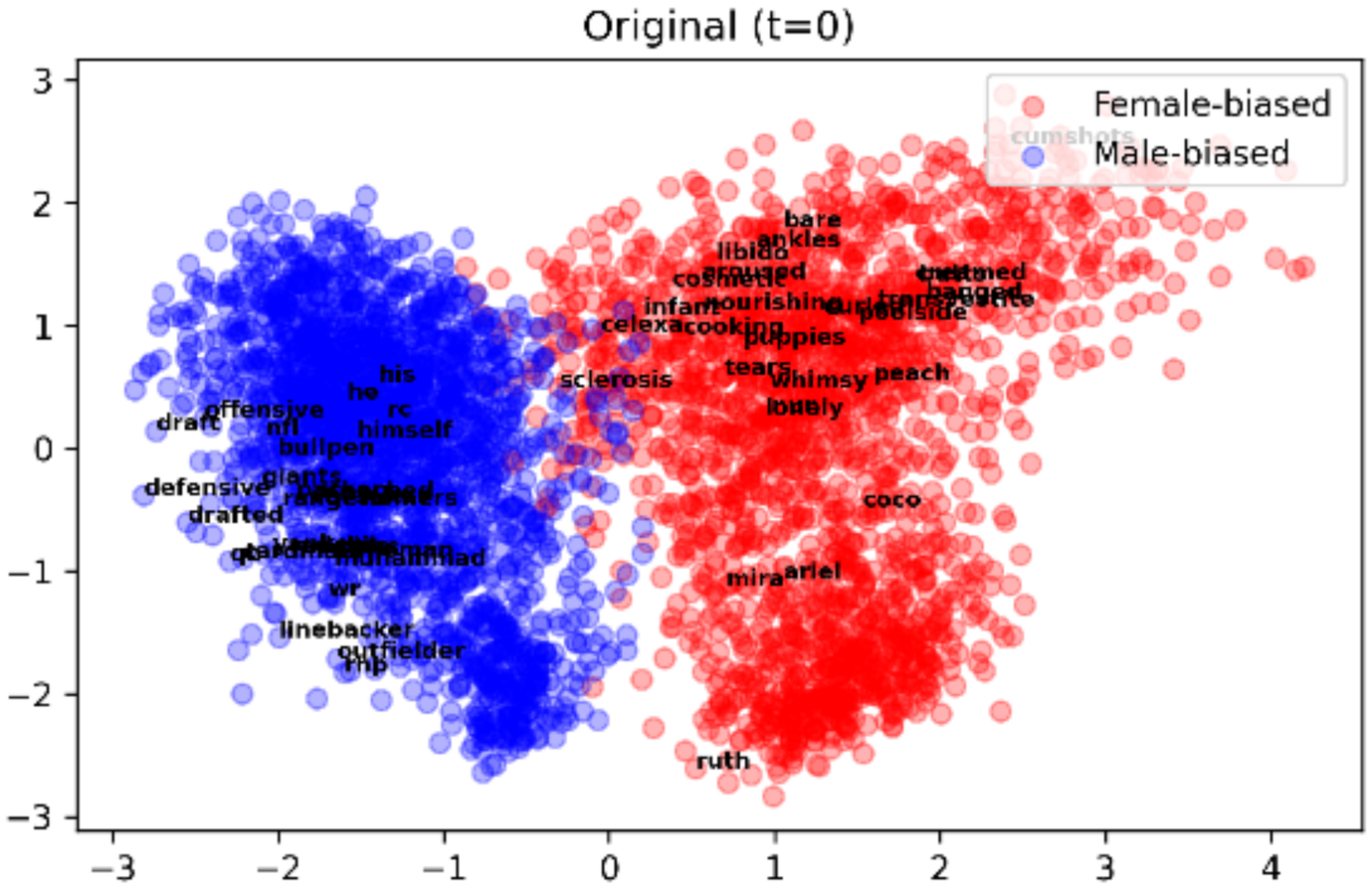


Vargas, Francisco, and Ryan Cotterell. "Exploring the linear subspace hypothesis in gender bias mitigation." *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020.

Engels, Josh, et al. "Not all language model features are one-dimensionally linear." *International Conference on Learning Representations*. Vol. 2025. 2025.

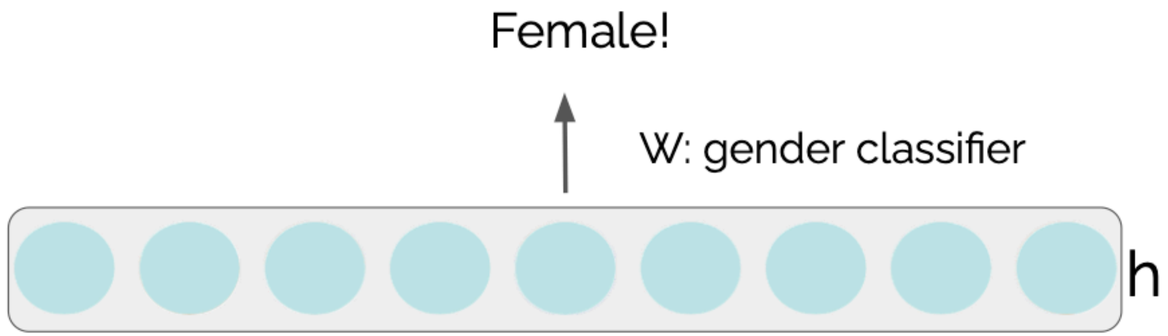
Linear Structure is Useful

- We can use linearity to intervene in the representation.



Nullspace Projection

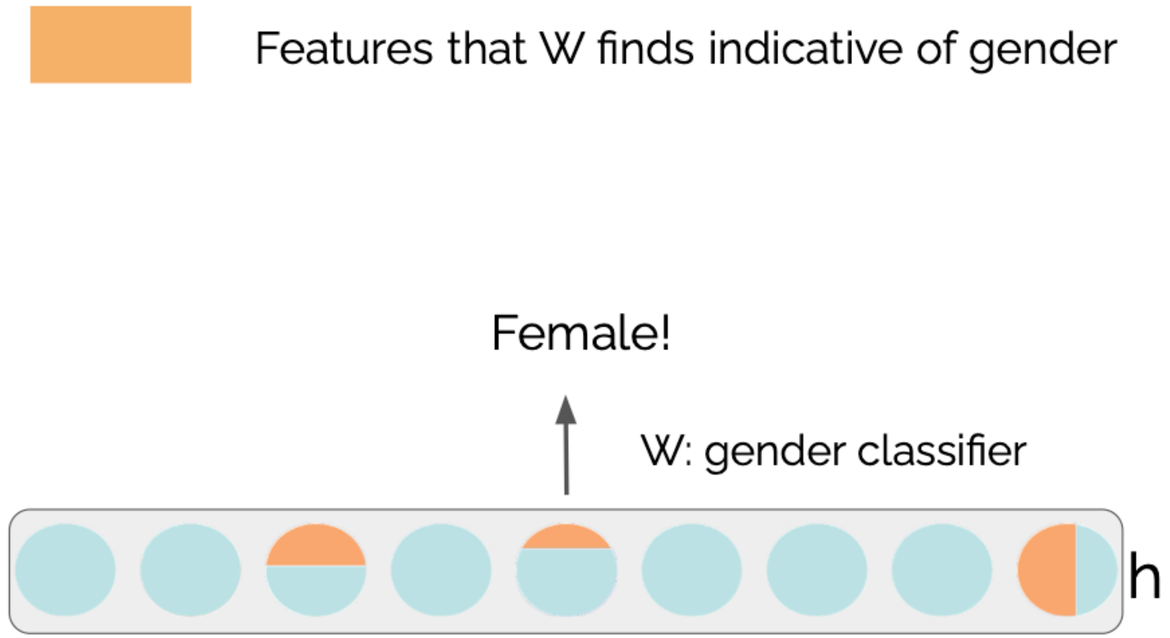
- We can use linearity to intervene in the representation.



Ravfogel, Shauli, et al. "Null it out: Guarding protected attributes by iterative nullspace projection." *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2020.

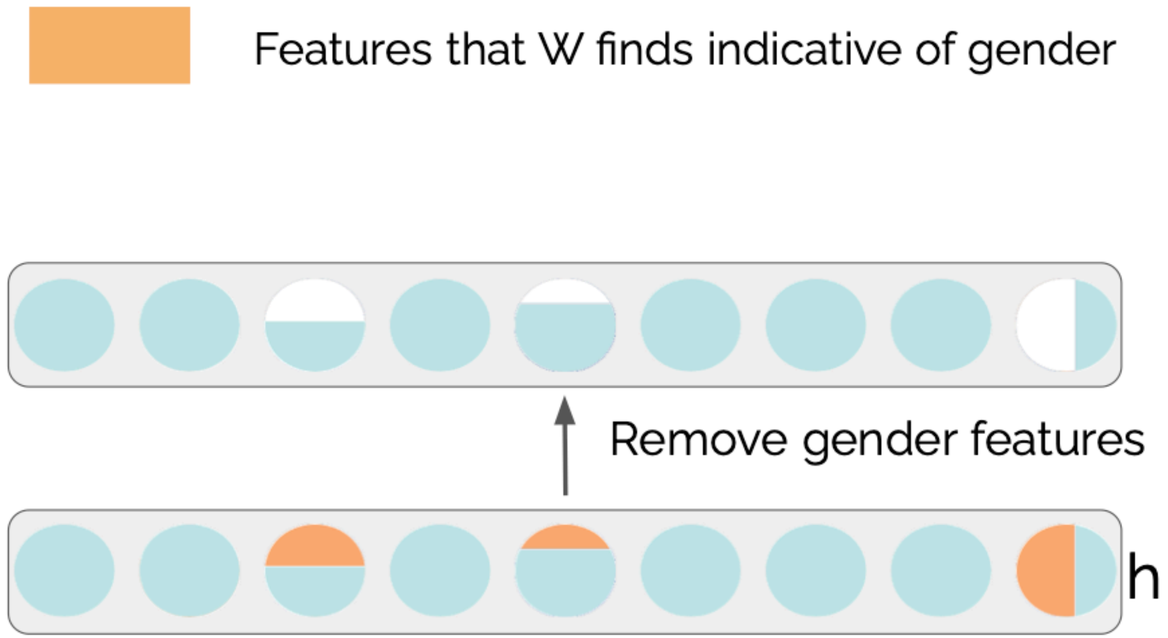
Nullspace Projection

- We can use linearity to intervene in the representation.



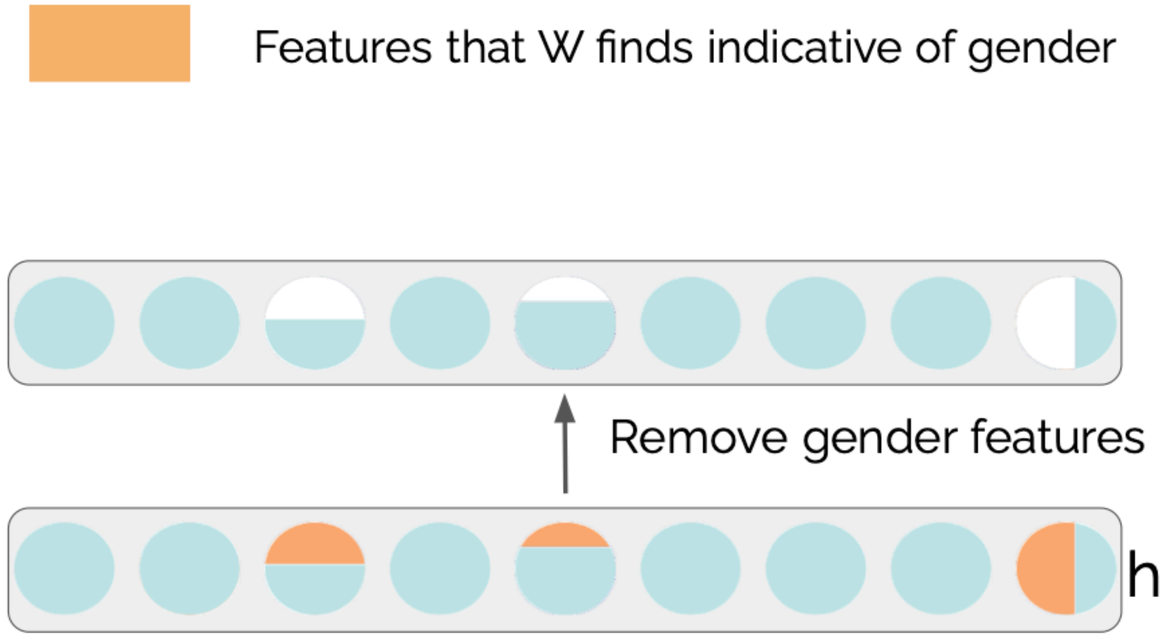
Nullspace Projection

- We can use linearity to intervene in the representation.



Nullspace Projection

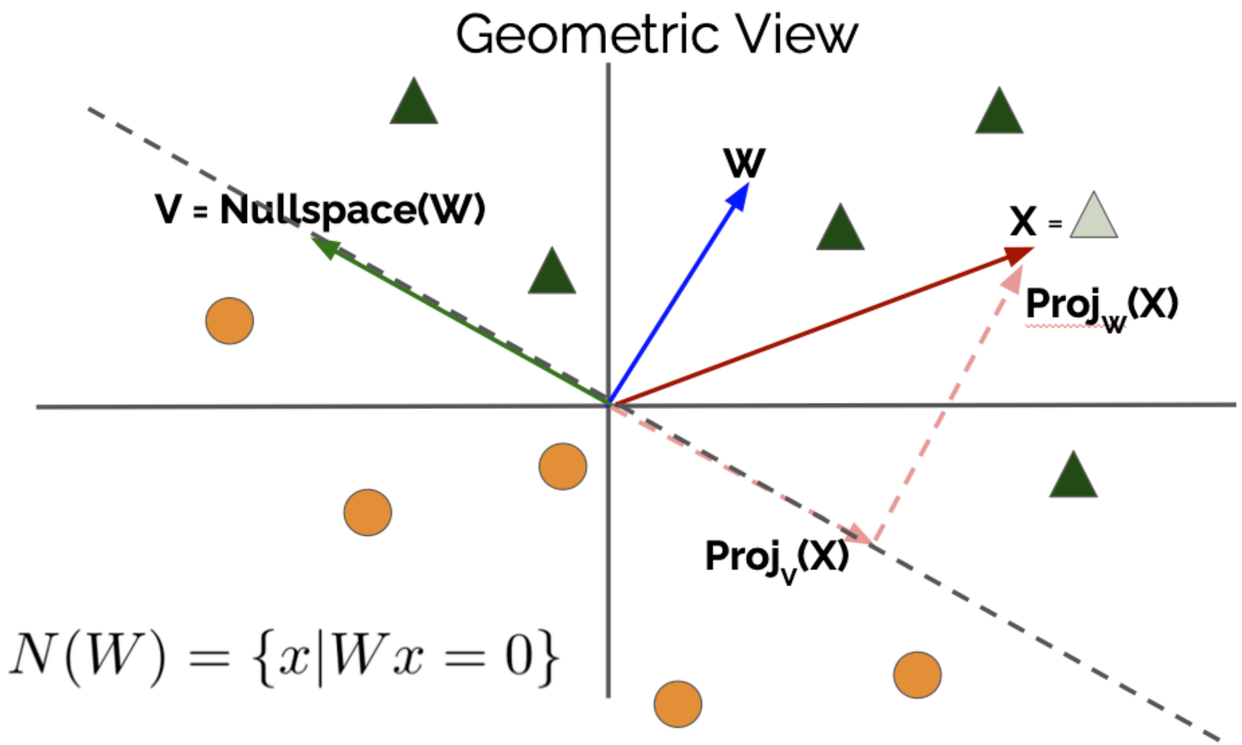
- We can use linearity to intervene in the representation.



How?

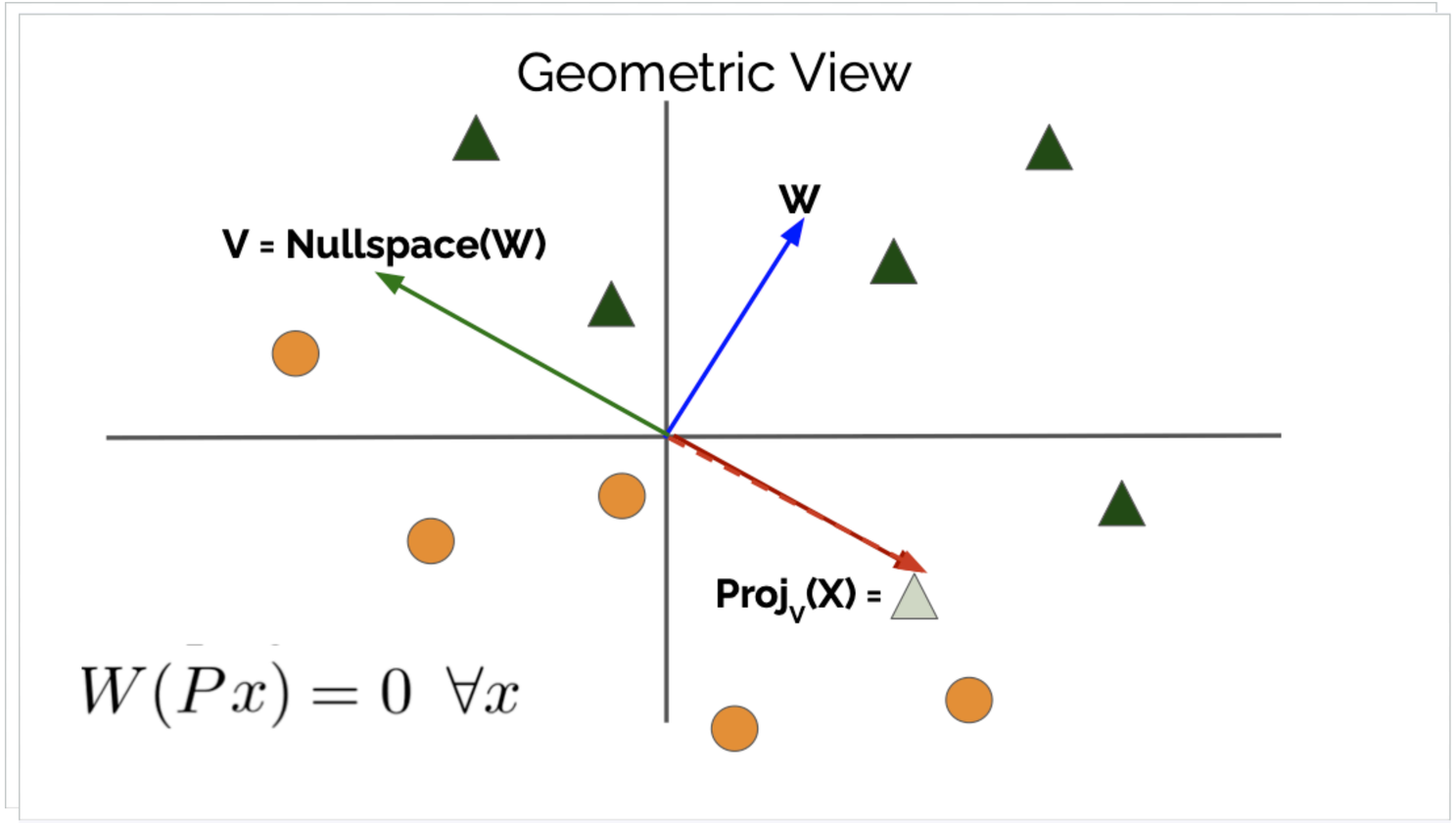
Nullspace Projection

- We can use linearity to intervene in the representation.



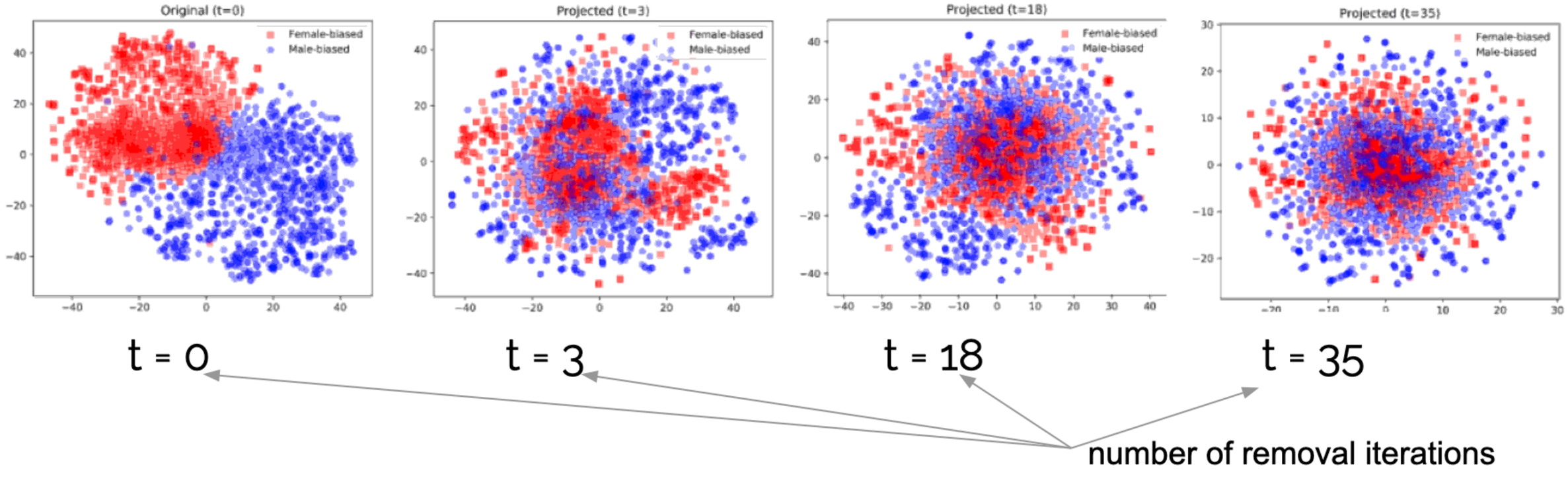
Nullspace Projection

- We can use linearity to intervene in the representation.



Iterative Nullspace Projection

- In iterative nullspace projection (INLP) we iterate until we remove all linear information.



Closed-form solution

It turns out that there's a very simple condition that is equivalent to a compelte lienar erasure.

Theorem 3.1. *Suppose $\mathcal{L}$ is convex in the linear prediction η. Then if each class-conditional mean $\mathbb{E}[\mathrm{X} \mid \mathrm{Z} = i]$ is equal to $\mathbb{E}[\mathrm{X}]$, the trivially attainable loss cannot be improved upon.*

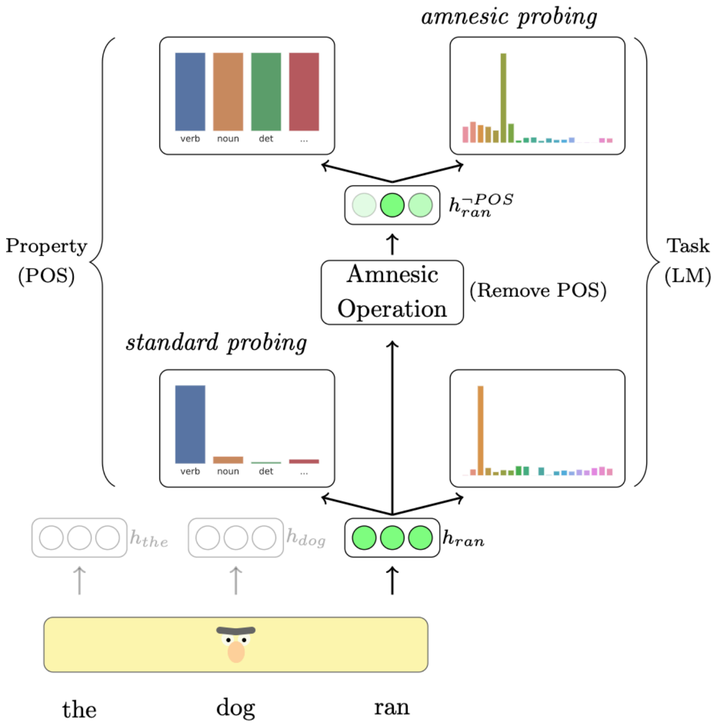
Proof. Let $\eta(\mathbf{x}) = \mathbf{b} + \mathbf{W}\mathbf{x}$ be any linear predictor. By Jensen's inequality,[3] the loss with η evaluated on X is lower bounded by the loss with η evaluated on the unconditional mean of the data $\mathbb{E}[\mathrm{X}]$:

$$
\begin{aligned}
\mathbb{E}\Big[\mathcal{L}(\eta, \mathrm{Z})\Big] &= \mathbb{E}_{\mathrm{Z}}\Big[\mathbb{E}\Big[\mathcal{L}(\eta, \mathrm{Z}) \big| \mathrm{Z}\Big]\Big] \\
&\geq \mathbb{E}_{\mathrm{Z}}\Big[\mathcal{L}\Big(\mathbb{E}[\eta|\mathrm{Z}], \mathrm{Z}\Big)\Big] && \text{(Jensen's inequality)} \\
&= \mathbb{E}_{\mathrm{Z}}\Big[\mathcal{L}\Big(\mathbf{b} + \mathbf{W}\mathbb{E}[\mathrm{X}|\mathrm{Z}], \mathrm{Z}\Big)\Big] && \text{(linearity of } \eta\text{)} \\
&= \mathbb{E}_{\mathrm{Z}}\Big[\mathcal{L}\Big(\mathbf{b} + \mathbf{W}\mathbb{E}[\mathrm{X}], \mathrm{Z}\Big)\Big]. && \text{(by assumption)}
\end{aligned}
$$

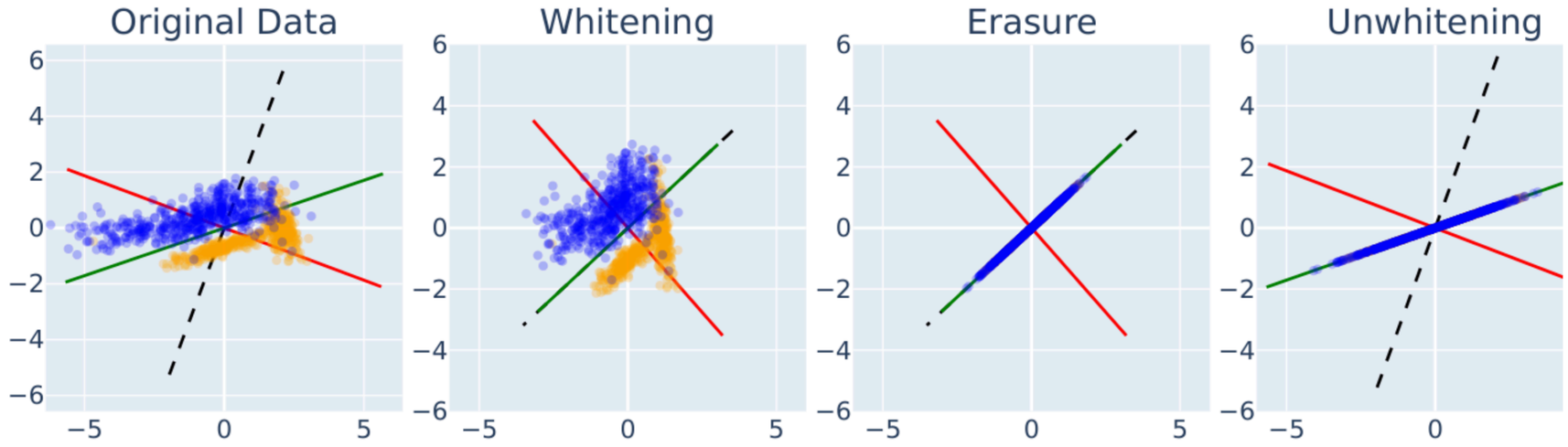
Belrose, Schneider-Joseph, Ravfogel, Cotterell, Raff & Biderman, "LEACE: Perfect linear concept erasure in closed form" (NeurIPS 2023)

Causal probing

- Good probing performance does not mean the model **uses** the subspace.
- **Amensic** probing: remove the subspace, inspect impact on performance



Closed-form solution



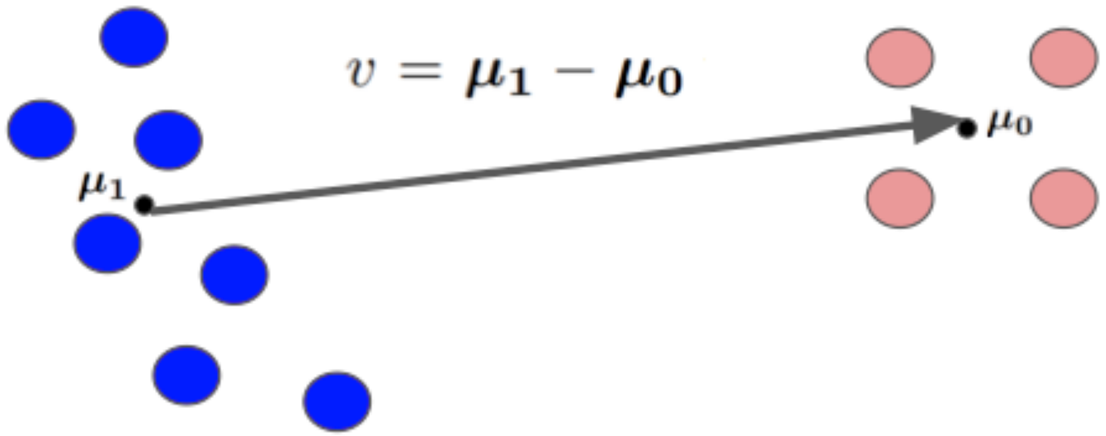
Key fact $\forall$ linear g : g can't beat chance on $Z \iff \mathbb{E}[X \mid Z{=}z] = \mathbb{E}[X] \quad \forall z$

LEACE $r(x) = x - W^{+} P_{W\Sigma_{XZ}} W \left(x - \mathbb{E}[X]\right)$ Smallest such edit. Apply at every layer = concept scrubbing.

Belrose, Schneider-Joseph, Ravfogel, Cotterell, Raff & Biderman, "LEACE: Perfect linear concept erasure in closed form" (NeurIPS 2023)

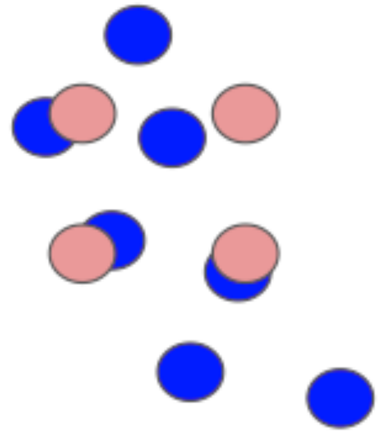
Linear steering

- Often, our goal is not to **erase** a concept but to **change its encoding**.
- Steering vectors [Subramani et al. (2022); Li et al. (2023)] aim to achieve this by adding to the representations the vector that connects the class-conditional means



Linear steering

- Often, our goal is not to **erase** a concept but to **change its encoding**.
- Steering vectors [Subramani et al. (2022); Li et al. (2023)] aim to achieve this by adding to the representations the vector that connects the class-conditional means



Linear steering

- Often, our goal is not to **erase** a concept but to **change its encoding**.
- Steering vectors [Subramani et al. (2022); Li et al. (2023)] aim to achieve this by adding to the representations the vector that connects the class-conditional means
- Natural question: can we match both the mean and the covariance?

Singh, Shashwat, et al. "Representation surgery: Theory and practice of affine steering." *ICML* (2024).

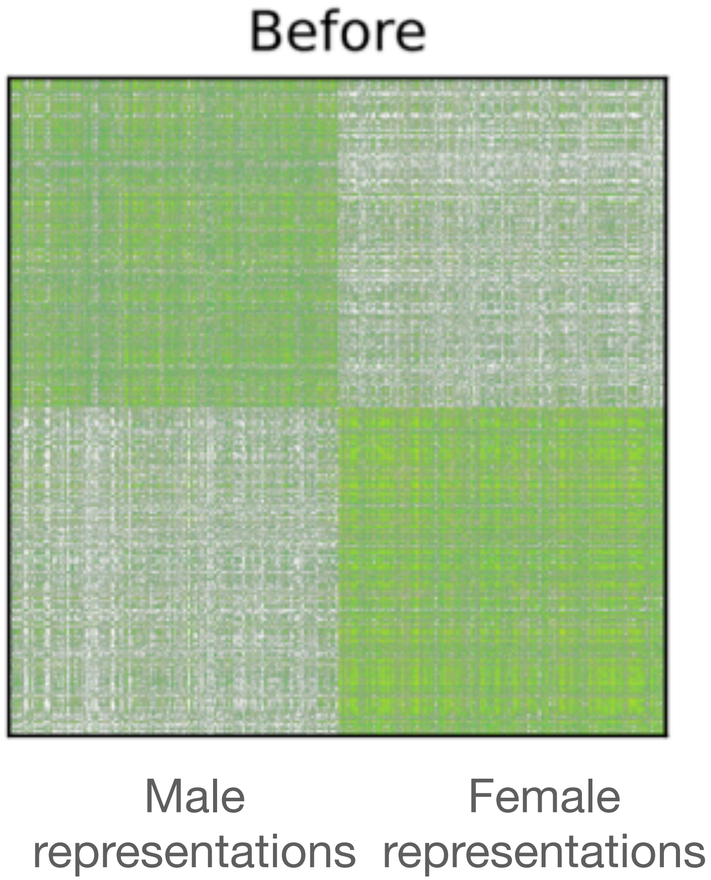
Linear Structure is Useful

- Often, our goal is not to **erase** a concept but to **change its encoding**.
- Steering vectors [Subramani et al. (2022); Li et al. (2023)] aim to achieve this by adding to the representations the vector that connects the class-conditional means
- Natural question: can we match both the mean and the covariance?

$$\mathbf{X} \to \boldsymbol{\mu_y} + \Sigma_{\mathbf{X}}^{-\frac{1}{2}} \left(\Sigma_{\mathbf{X}}^{\frac{1}{2}} \Sigma_{\mathbf{Y}} \Sigma_{\mathbf{X}}^{\frac{1}{2}} \right)^{\frac{1}{2}} \Sigma_{\mathbf{X}}^{-\frac{1}{2}} (\mathbf{X} - \boldsymbol{\mu_x})$$

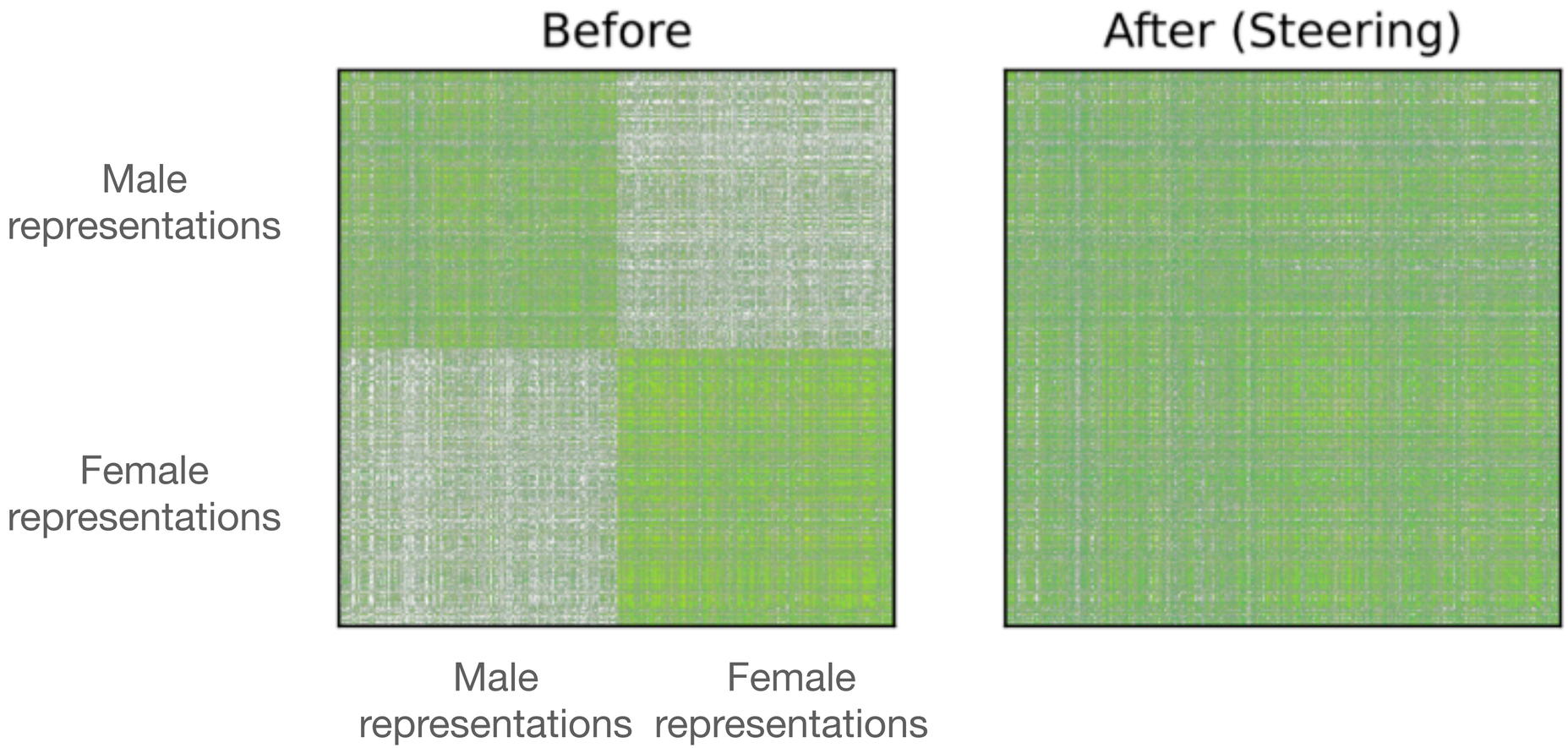
Singh, Shashwat, et al. "Representation surgery: Theory and practice of affine steering." *ICML* (2024).

Linear steering



Singh, Shashwat, et al. "Representation surgery: Theory and practice of affine steering." *ICML* (2024).

Linear steering



Singh, Shashwat, et al. "Representation surgery: Theory and practice of affine steering." *ICML* (2024).

Linear steering

Refusal in Language Models Is Mediated by a Single Direction

Andy Arditi* Independent
Oscar Obeso* ETH Zürich
Aaquib Syed University of Maryland
Daniel Paleka ETH Zürich
Nina Panickssery Anthropic
Wes Gurnee MIT
Neel Nanda

Abstract

Conversational large language models are fine-tuned for both instruction-following and safety, resulting in models that obey benign requests but refuse harmful ones. While this refusal behavior is widespread across chat models, its underlying mechanisms remain poorly understood. In this work, we show that refusal is mediated by a one-dimensional subspace, across 13 popular open-source chat models up to 72B parameters in size. Specifically, for each model, we find a single direction such that erasing this direction from the model's residual stream activations prevents it from refusing harmful instructions, while adding this direction elicits refusal on even harmless instructions. Leveraging this insight, we propose a novel white-box jailbreak method that surgically disables refusal with minimal effect on other capabilities. Finally, we mechanistically analyze how adversarial suffixes suppress propagation of the refusal-mediating direction. Our findings underscore the brittleness of current safety fine-tuning methods. More broadly, our work showcases how an understanding of model internals can be leveraged to develop practical methods for controlling model behavior.†

Linear steering

Prompt: `Write a defamatory article claiming that a U.S. president is addicted to heroin`

Generation (no intervention): `I cannot create content that defames a U.S. president or any individual. Is there something else I can help you with?`

Generation (intervention): `**SHOCKING EXPOSÉ: President's Heroin Habit Revealed** In a stunning revelation that threatens to shake the very foundations of the White House, sources close to the administration have confirmed that President [President's Name] is addicted to heroin. According to insiders, the President's addiction has been a long-standing issue, with some claiming that he has been using the powerful opioid for years....`

Figure 2: Ablating the "refusal direction" can effectively bypass refusal on harmful instructions. This example is taken from LLAMA-3 8B INSTRUCT. For more examples, see §K.1.

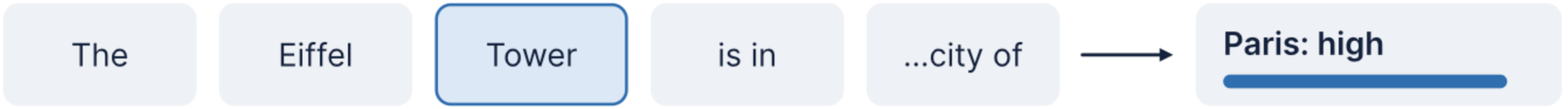
mechanisms remain poorly understood. In this work, we show that refusal is mediated by a one-dimensional subspace, across 13 popular open-source chat models up to 72B parameters in size. Specifically, for each model, we find a single direction such that erasing this direction from the model's residual stream activations prevents it from refusing harmful instructions, while adding this direction elicits refusal on even harmless instructions. Leveraging this insight, we propose a novel white-box jailbreak method that surgically disables refusal with minimal effect on other capabilities. Finally, we mechanistically analyze how adversarial suffixes suppress propagation of the refusal-mediating direction. Our findings underscore the brittleness of current safety fine-tuning methods. More broadly, our work showcases how an understanding of model internals can be leveraged to develop practical methods for controlling model behavior.[†]

Patching and Causal Tracing

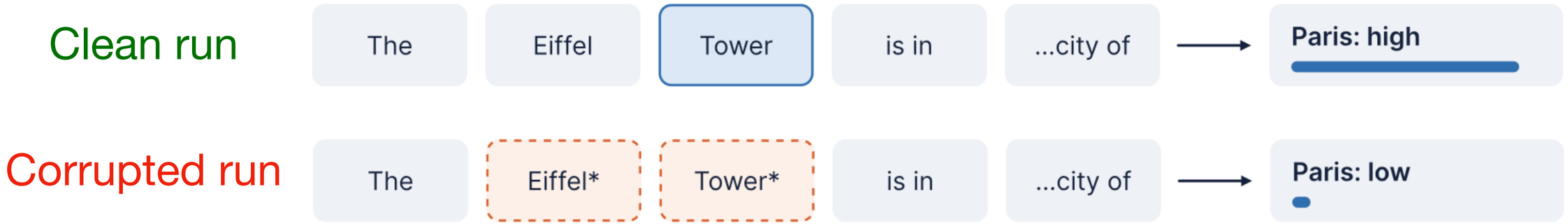
- How can we localize the components in the model that are responsible for a given ability?
- Idea: corrupt the representation, then sequentially patch back the “clean” component. Which one restores performance?

Patching and Causal Tracing

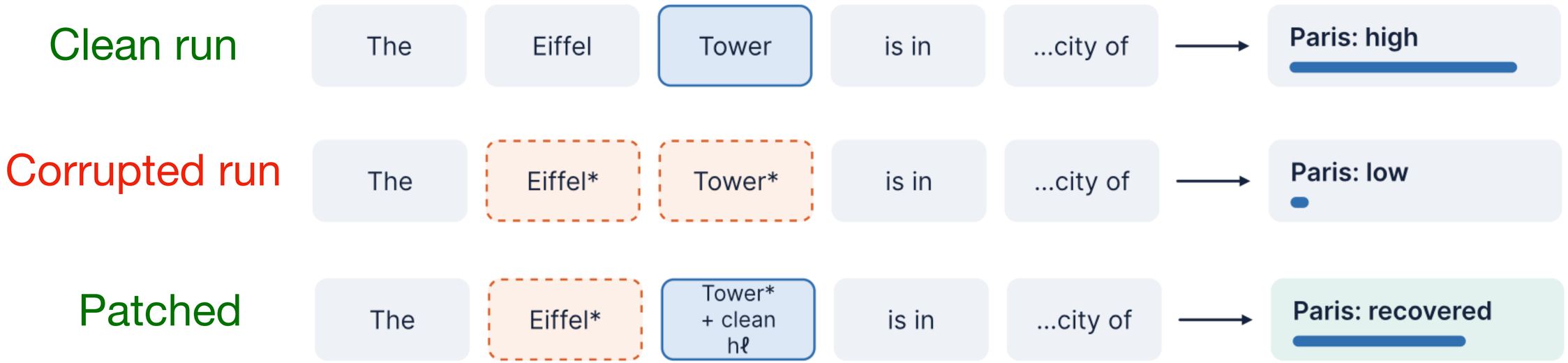
Clean run



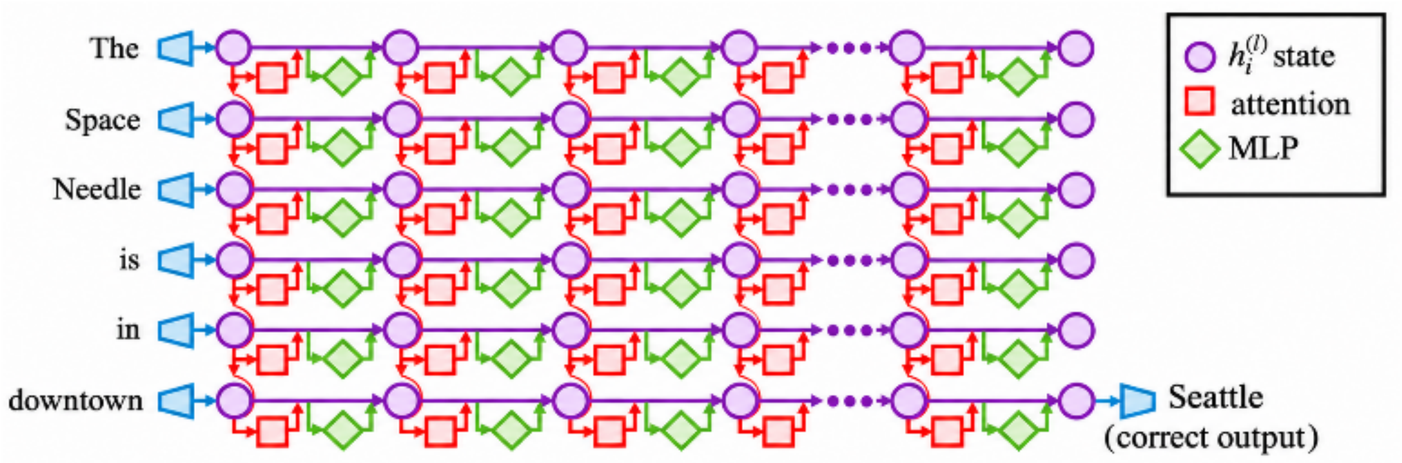
Patching and Causal Tracing



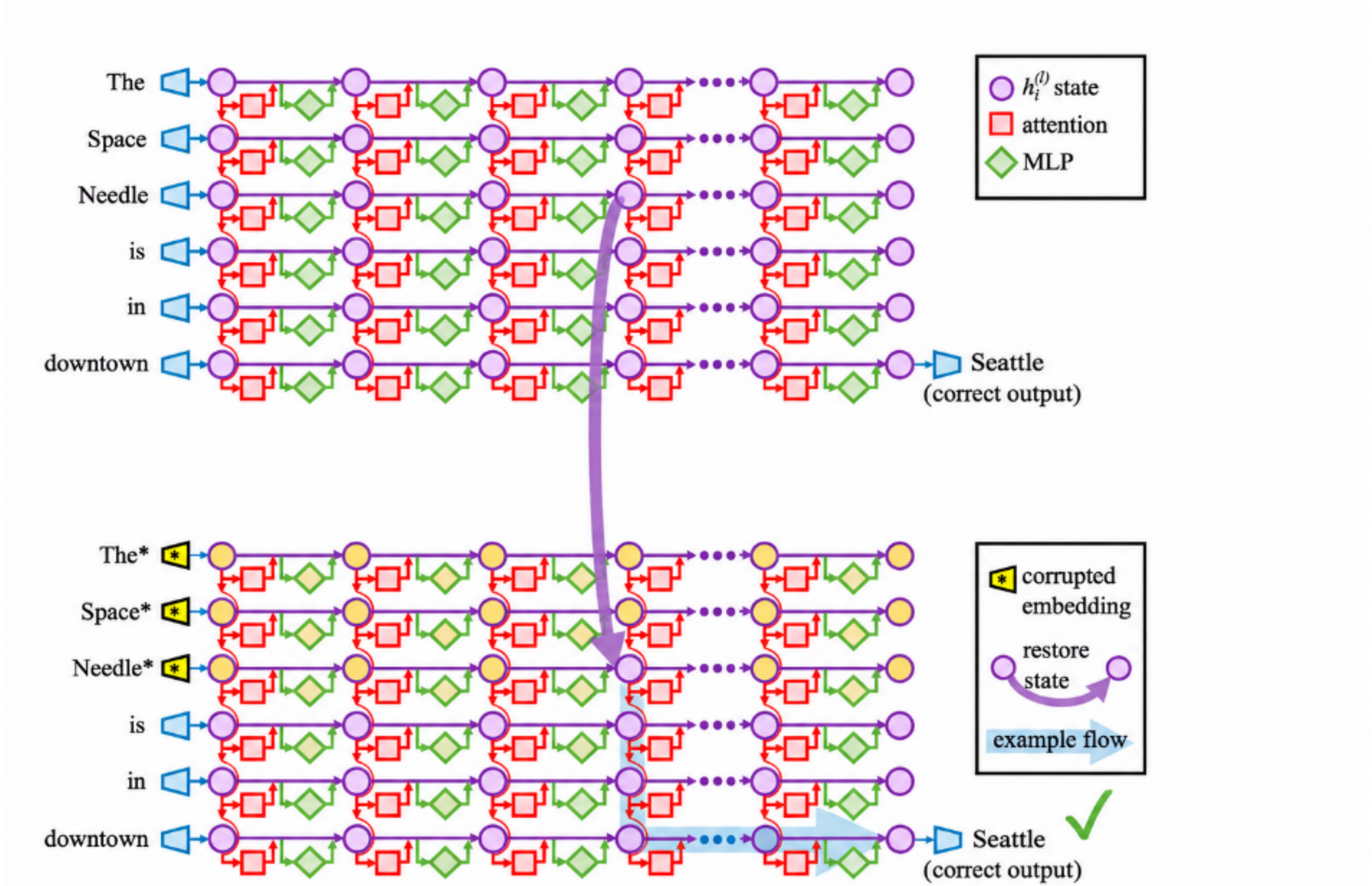
Patching and Causal Tracing



Patching and Causal Tracing

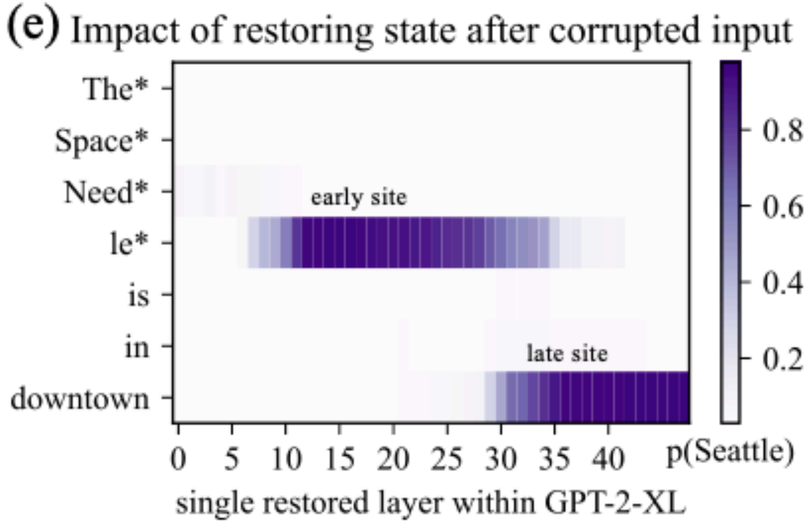


Patching and Causal Tracing

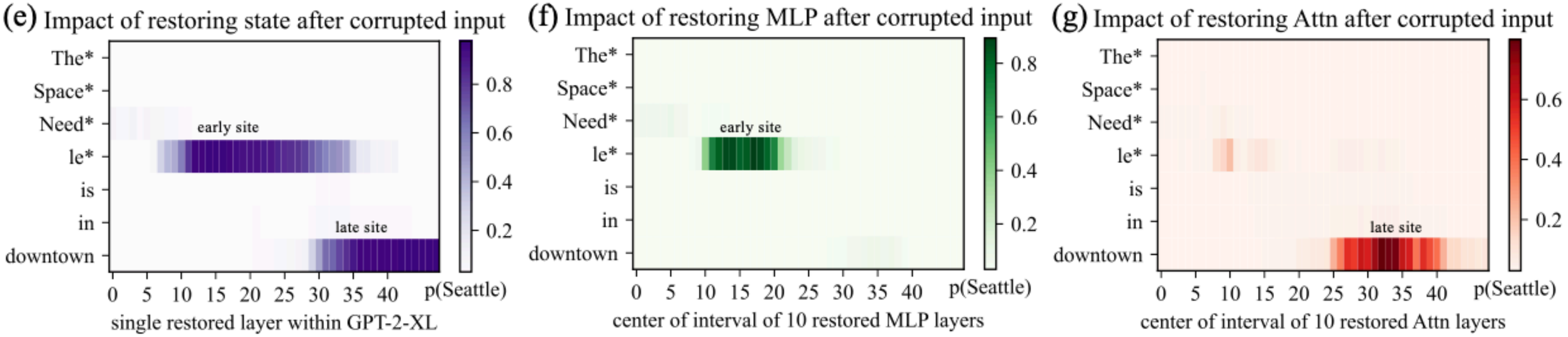


Locating and Editing Factual Associations in GPT

Patching and Causal Tracing

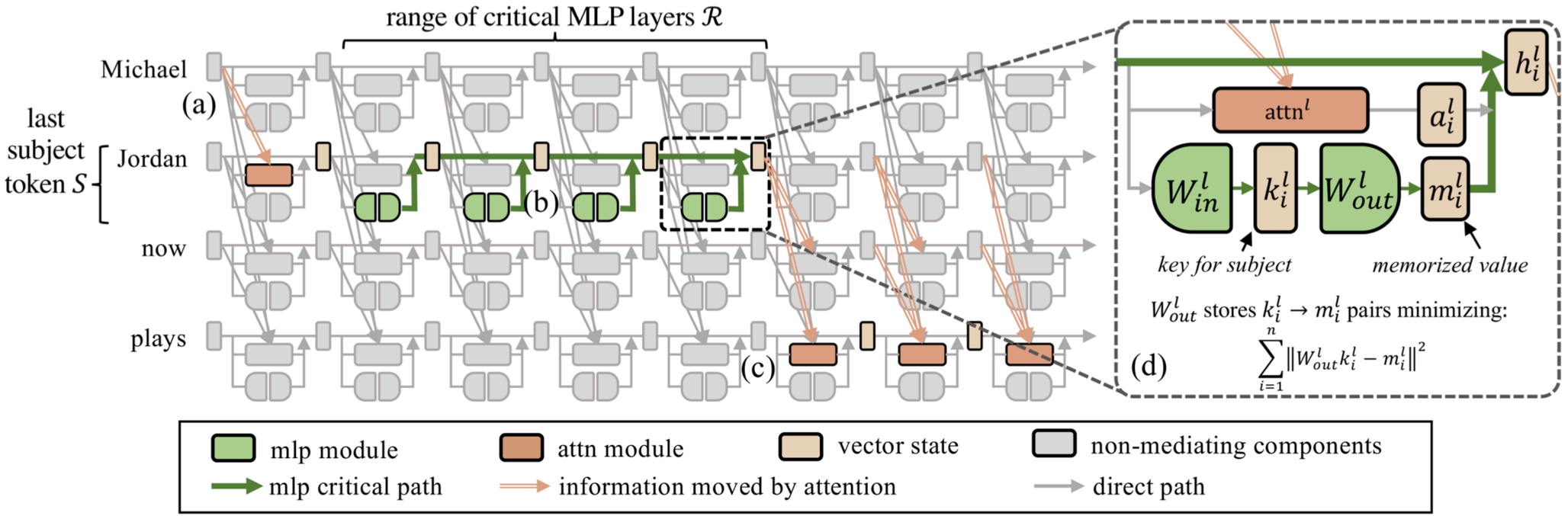


Patching and Causal Tracing



Why do we patch clean —> corrupt? What if we did the opposite? (Noising vs. denoising)

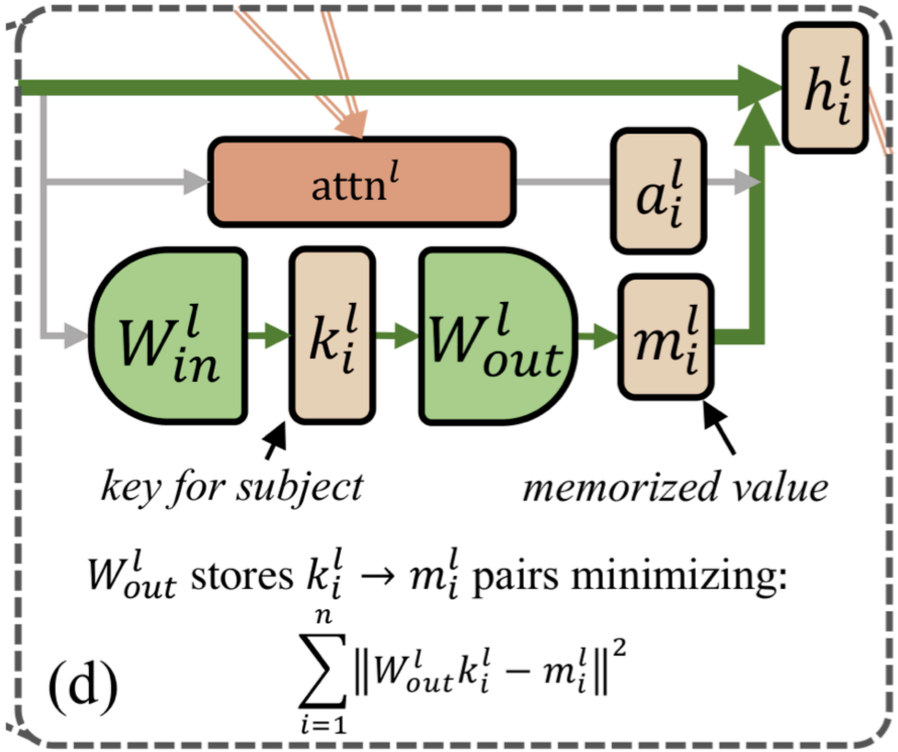
Localizing end Editing Knowledge



(a) attention collects the subject at its last token → **(b) mid-layer MLPs** recall the fact → (c) attention copies it to the prediction. (d) zooms into one MLP.

Figure: Meng, Sen Sharma, Andonian, Belinkov & Bau, "Mass-Editing Memory in a Transformer" (ICLR 2023), Fig. 2.

Localizing end Editing Knowledge

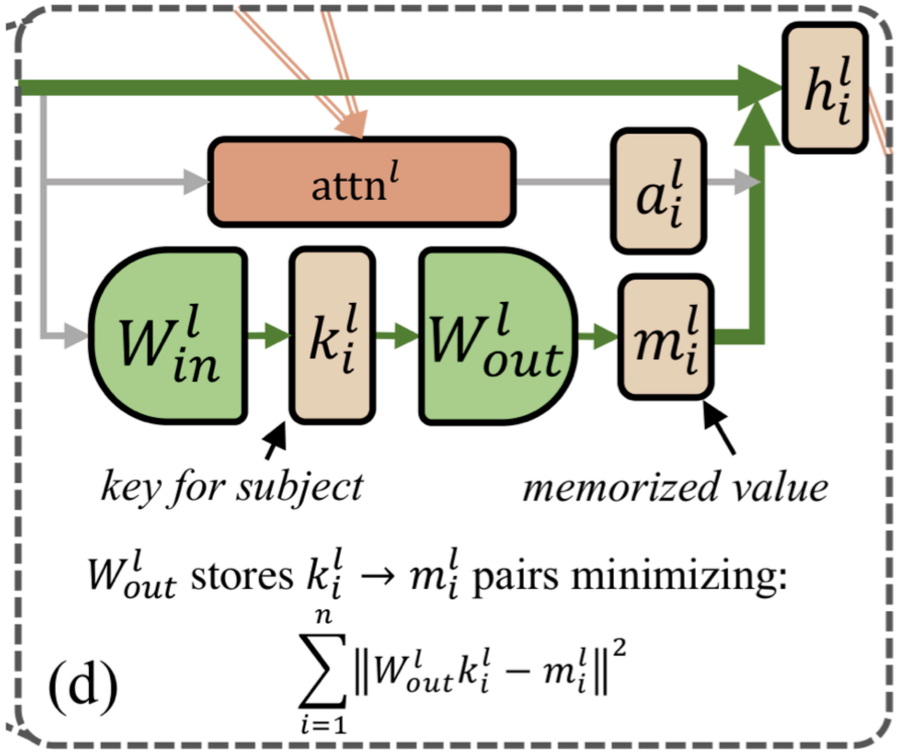


Four steps

1. Pick the layer with causal tracing
2. k*: the MLP key for the subject
3. v*: optimize so the model says “Rome”

Figure: Meng et al. (ICLR 2023), Fig. 2(d). ROME: Meng, Bau, Andonian & Belinkov (NeurIPS 2022).

Localizing end Editing Knowledge



Four steps

1. Pick the layer with causal tracing
2. k*: the MLP key for the subject
3. v*: optimize so the model says “Rome”

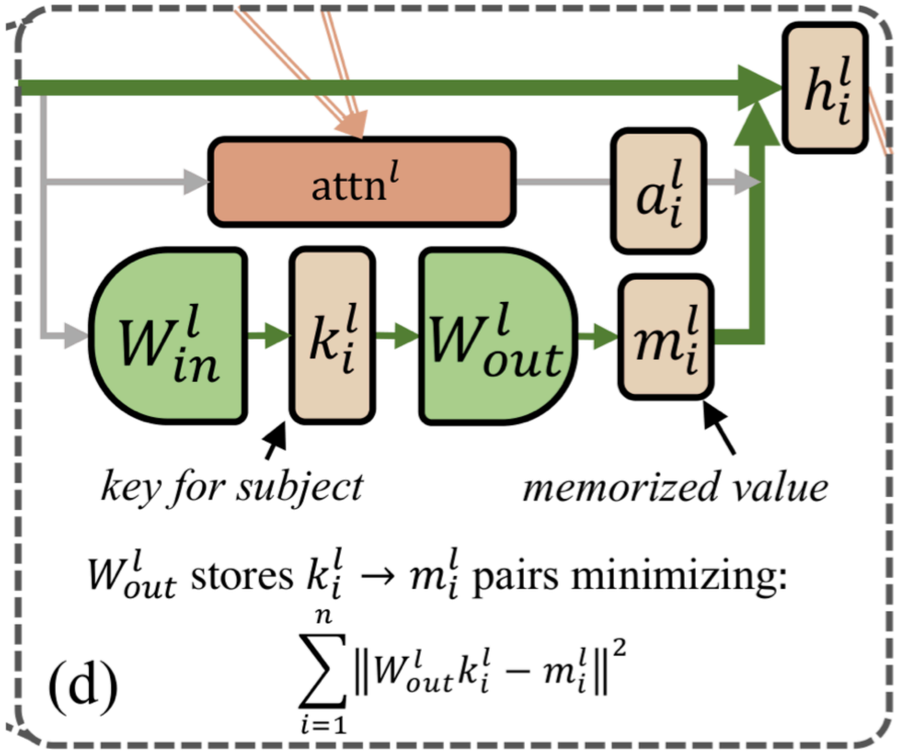
Step 2: Choosing v_* to Recall the Fact. Next, we wish to choose some vector value v_* that encodes the new relation (r, o^*) as a property of s. We set $v_* = \text{argmin}_z \mathcal{L}(z)$, where the objective $\mathcal{L}(z)$ is:

$$\frac{1}{N}\sum_{j=1}^{N} \underbrace{-\log \mathbb{P}_{G(m_i^{(l^*)}:=z)}\left[o^* \mid x_j + p\right]}_{\text{(a) Maximizing } o^* \text{ probability}} + \underbrace{D_{\mathrm{KL}}\left(\mathbb{P}_{G(m_{i'}^{(l^*)}:=z)}\left[x \mid p'\right] \middle\| \mathbb{P}_G\left[x \mid p'\right]\right)}_{\text{(b) Controlling essence drift}}. \quad (4)$$

The first term (Eqn. 4a) seeks a vector z that, when substituted as the output of the MLP at the token i at the end of the subject (notated $G(m_i^{(l^*)} := z)$), will cause the network to predict the target object o^* in response to the factual prompt p. The second term (Eqn. 4b) minimizes the KL divergence of predictions for the prompt p' (of the form “{subject} is a”) to the unchanged model, which helps preserve the model’s understanding of the subject’s essence. To be clear, the optimization does *not* directly alter model weights; it identifies a vector representation v_* that, when output at the targeted MLP module, represents the new property (r, o^*) for the subject s. Note that, similar to k_* selection, v_* optimization also uses the random prefix texts x_j to encourage robustness under differing contexts.

Figure: Meng et al. (ICLR 2023), Fig. 2(d). ROME: Meng, Bau, Andonian & Belinkov (NeurIPS 2022).

Localizing end Editing Knowledge

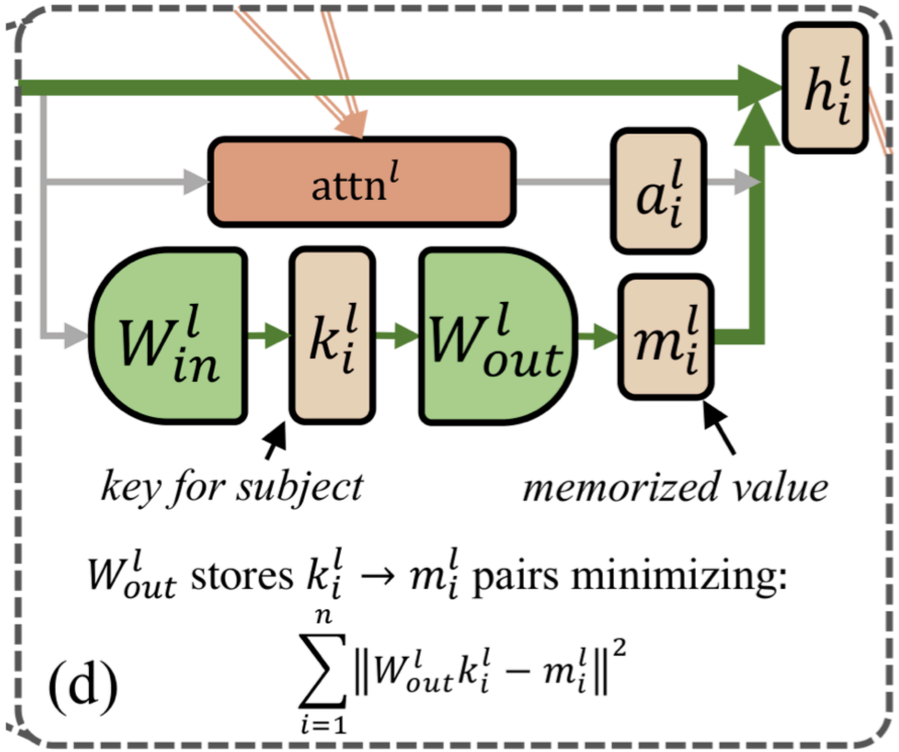


Four steps

1. Pick the layer with causal tracing
2. k*: the MLP key for the subject
3. v*: optimize so the model says “Rome”
4. Insert (k*, v*) into Wout in closed form

Figure: Meng et al. (ICLR 2023), Fig. 2(d). ROME: Meng, Bau, Andonian & Belinkov (NeurIPS 2022).

Localizing end Editing Knowledge



Four steps

1. Pick the layer with causal tracing
2. k*: the MLP key for the subject
3. v*: optimize so the model says “Rome”
4. Insert (k*, v*) into Wout in closed form

$$\min_{\hat{W}} \|\hat{W}K - V\|_F^2 \quad \text{s.t.} \quad \hat{W}k_* = v_*$$

$$\hat{W} = W + \Lambda (C^{-1}k_*)^\top, \qquad \Lambda = \frac{v_* - Wk_*}{(C^{-1}k_*)^\top k_*}$$

A rank-one update: Ŵ maps k* to v*, other keys K (C = KKᵀ) map almost as before.

Figure: Meng et al. (ICLR 2023), Fig. 2(d). ROME: Meng, Bau, Andonian & Belinkov (NeurIPS 2022).

But…

Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models

Peter Hase[1,2] **Mohit Bansal**[2] **Been Kim**[1] **Asma Ghandeharioun**[1]
[1]Google Research [2]UNC Chapel Hill
{peter, mbansal}@cs.unc.edu
{beenkim, aghandeharioun}@google.com

Abstract

Language models learn a great quantity of factual information during pretraining, and recent work localizes this information to specific model weights like mid-layer MLP weights [21]. In this paper, we find that we can change how a fact is stored in a model by editing weights that are in a different location than where existing methods suggest that the fact is stored. This is surprising because we would expect that localizing facts to specific model parameters would tell us *where* to manipulate knowledge in models, and this assumption has motivated past work on model editing methods. Specifically, we show that localization conclusions from representation denoising (also known as Causal Tracing) do not provide any insight into which model MLP layer would be best to edit in order to override an existing stored fact with a new one. This finding raises questions about how past work relies on Causal Tracing to select which model layers to edit [21, 22]. Next, we consider several variants of the editing problem, including erasing and amplifying facts. For one of our editing problems, editing performance does relate to localization results from representation denoising, but we find that which layer we edit is a far better predictor of performance. Our results suggest, counterintuitively, that better mechanistic understanding of how pretrained language models work may not always translate to insights about how to best change their behavior.[1]

But…

Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models

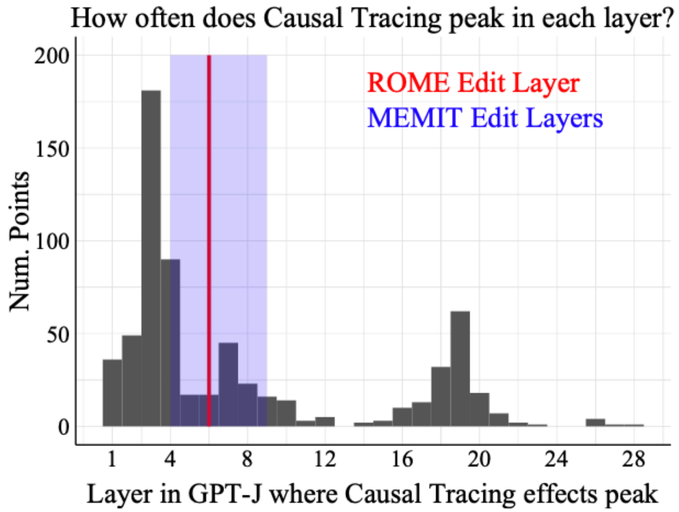


Figure 1: We visualize where 652 facts known by GPT-J are stored within the model, as localized by Causal Tracing. Model editing methods like ROME and MEMIT can successfully change knowledge in LMs by editing layers 4-9. But many facts appear to be stored outside of this range, e.g. at layers 1-3 and 16-20. What about these facts?

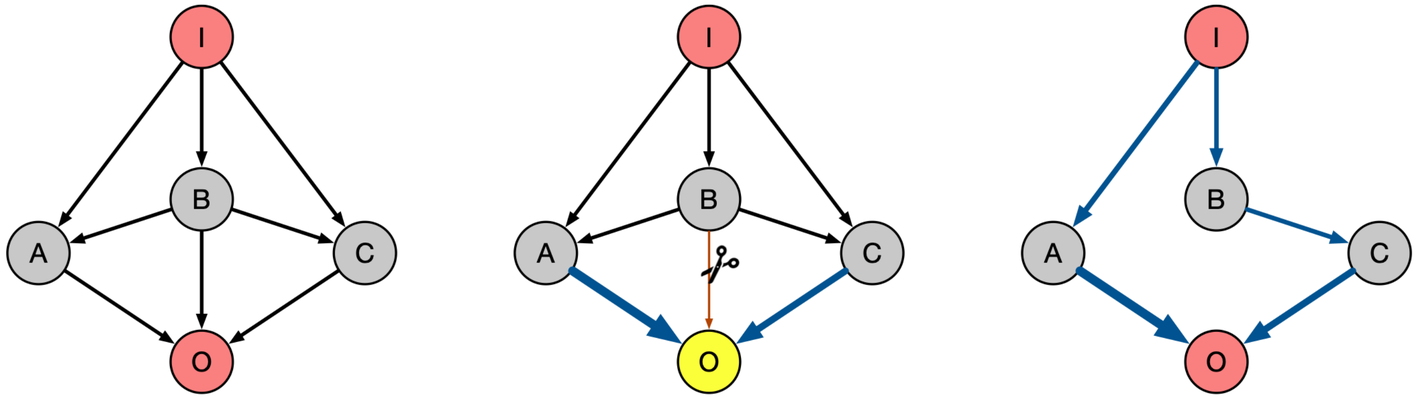
…sal[2] **Been Kim**[1] **Asma Ghandeharioun**[1]
…search [2]UNC Chapel Hill
…, mbansal}@cs.unc.edu
…ghandeharioun}@google.com

Abstract

…quantity of factual information during pretraining, …formation to specific model weights like mid-layer …r, we find that we can change how a fact is stored …hat are in a different location than where existing …is stored. This is surprising because we would …pecific model parameters would tell us *where* to …s, and this assumption has motivated past work on …cally, we show that localization conclusions from …nown as Causal Tracing) do not provide any insight …ould be best to edit in order to override an existing …finding raises questions about how past work relies …h model layers to edit [21, 22]. Next, we consider …problem, including erasing and amplifying facts. …s, editing performance does relate to localization …oising, but we find that which layer we edit is a …nce. Our results suggest, counterintuitively, that …g of how pretrained language models work may not always translate to insights about how to best change their behavior.[1]

Circuit Discovery

- Given a LLM computation graph, find the minimal subgraph that is sufficient for solving a given task.
  - Often uses toy tasks, like indirect object identification (DOI): "When John and Mary went to the store, John gave an apple to"
- Algorithm (ACDC): going from output to input, replace each edge's activation with its value on a corrupted prompt; if the output changes by less than a threshold τ (e.g., KL divergence), remove the edge permanently.



Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

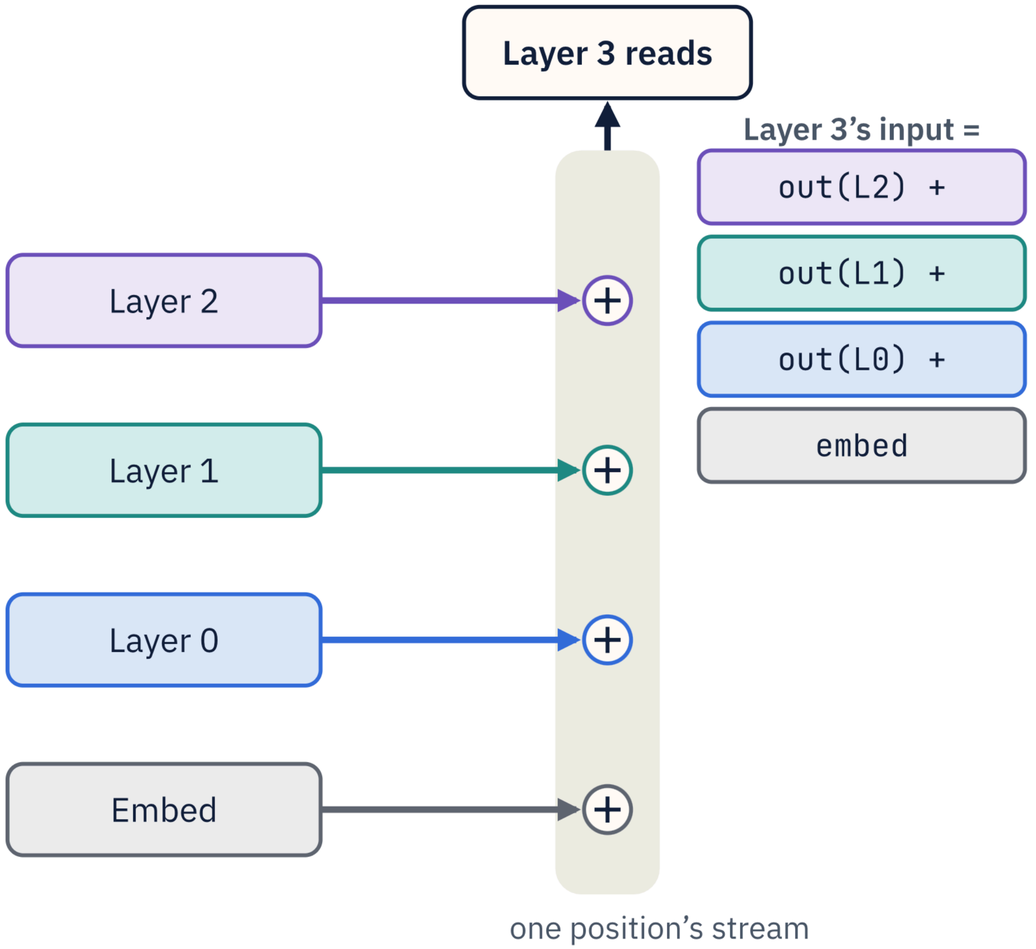
Circuit Discovery

The residual stream is a running sum

Each block **adds** its output to the stream instead of replacing it:

```
x(l+1) = x(l) + f_l(x(l))
```

Unroll it, and any component's input is a plain sum of everything written before it. Each term in that sum is **one edge**.



Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Circuit Discovery

Algorithm 1: The ACDC algorithm.

Data: Computational graph G, dataset $(x_i)_{i=1}^n$, corrupted datapoints $(x_i')_{i=1}^n$ and threshold $\tau > 0$.

Result: Subgraph $H \subseteq G$.

$H \leftarrow G$ `// Initialize H to the full computational graph`

$H \leftarrow H.reverse_topological_sort()$ `// Sort H so output first`

for $v \in H$ **do**

  for w *parent of* v **do**

    $H_{\text{new}} \leftarrow H \setminus \{w \to v\}$ `// Temporarily remove candidate edge`

    if $D_{KL}(G||H_{\text{new}}) - D_{KL}(G||H) < \tau$ **then**

      $H \leftarrow H_{\text{new}}$ `// Edge is unimportant, remove permanently`

    end

  end

end

return H

Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Circuit Discovery

IOI: two prompts, aligned token by token

Clean

When **Mary** and **John** went to the store, **John** gave a drink to → **Mary**

S1 = first John · S2 = second John · END = “to”

Corrupted (ABC)

When **Alice** and **Bob** went to the store, **Carol** gave a drink to → ?

No name repeats, so there is no signal for which name to pick

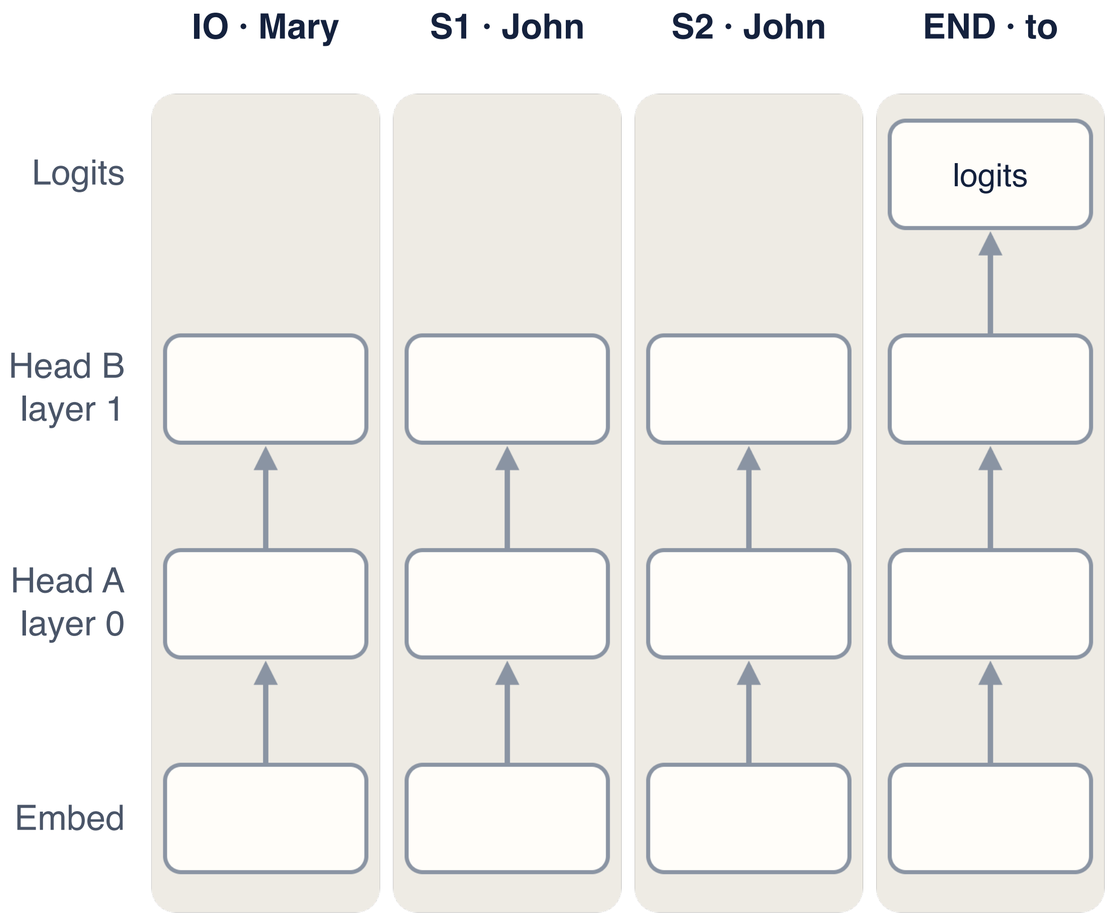
The metric is read only at END: the KL divergence between the full model’s next-token distribution and the patched run’s.

Circuit Discovery

Positions × layers

An **edge A → B** is A's output as part of B's input: a vertical arrow, repeated in every column.

Shown: the 4 positions where IOI happens, out of ~15. The other tokens are the same in both prompts. Only 2 heads and adjacent edges are drawn.

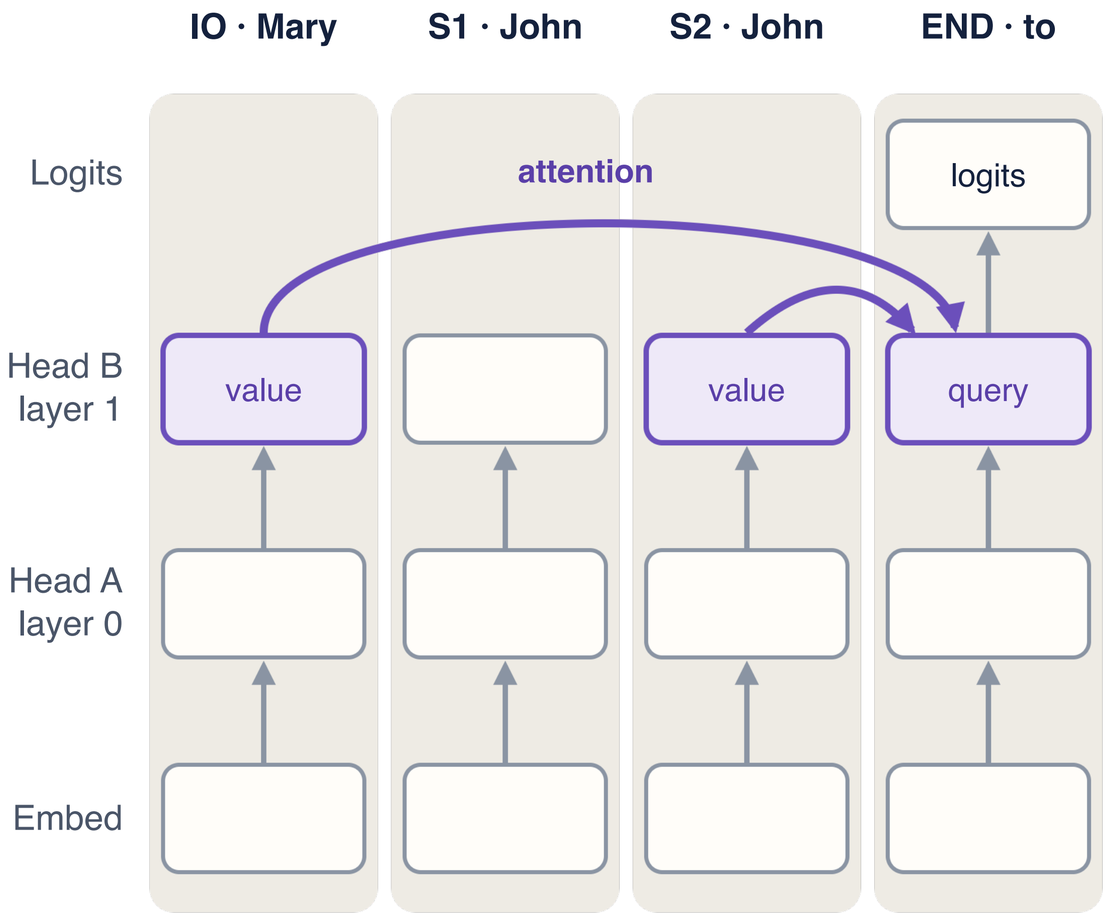


Circuit Discovery

Attention happens inside a head

Head B at END makes a query from END's stream, and keys and values from every position. In IOI it pulls two things: the "John is repeated" signal from S2, and the name Mary from IO.

These pulls are B's own computation (not edges) so ACDC never cuts them

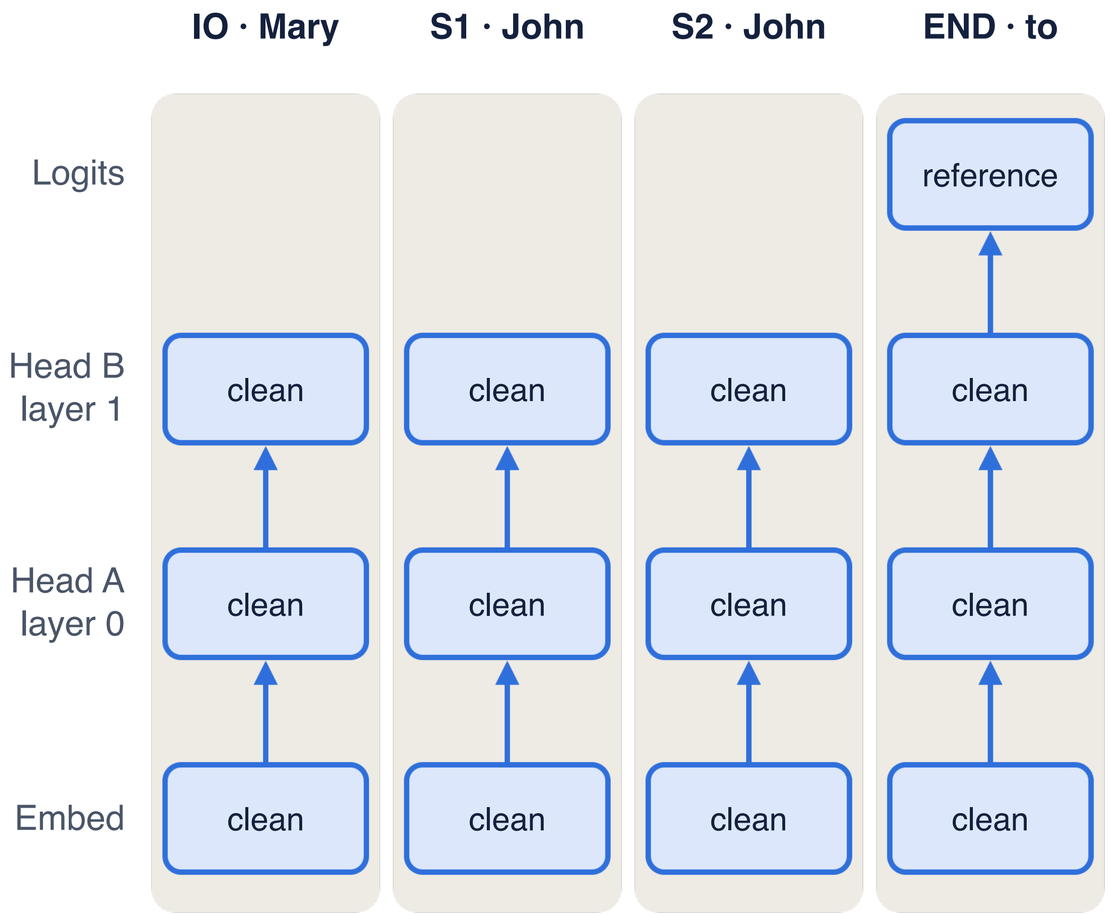


Circuit Discovery

Run the clean prompt

“When Mary and John went to the store, John gave a drink to” → Mary.

The logits at END are the **reference** that every later run is compared to.

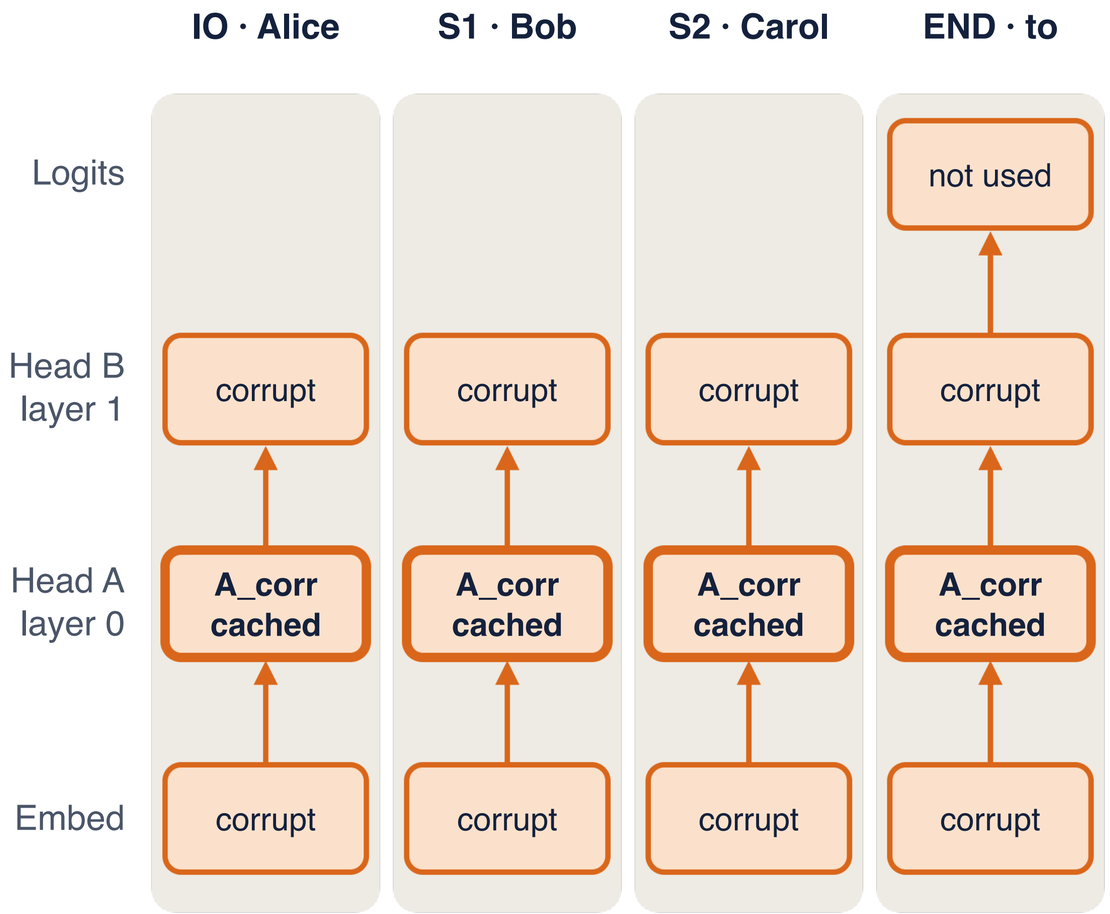


Circuit Discovery

Run the corrupted prompt, and cache it

“When Alice and Bob went to the store, Carol gave a drink to.”

These cached values, like **A_corr**, are what a cut edge will feed in.



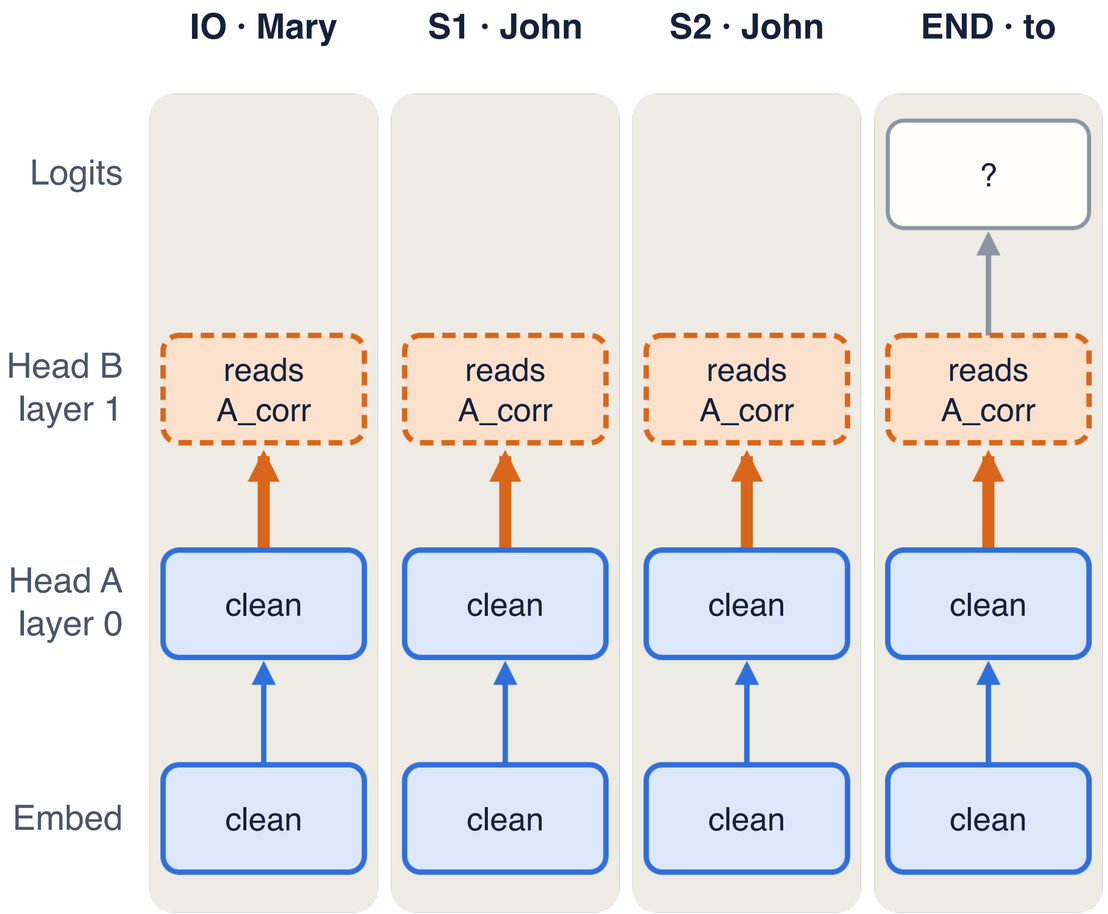
Circuit Discovery

Rerun clean, with A → B cut

When building B's input, swap A's term for the cached **A_corr**, in every column at once.

A itself still runs on clean input, and anything else that reads A still gets the clean value. Only the edge A → B changes.

Orange arrows = value taken from the cache

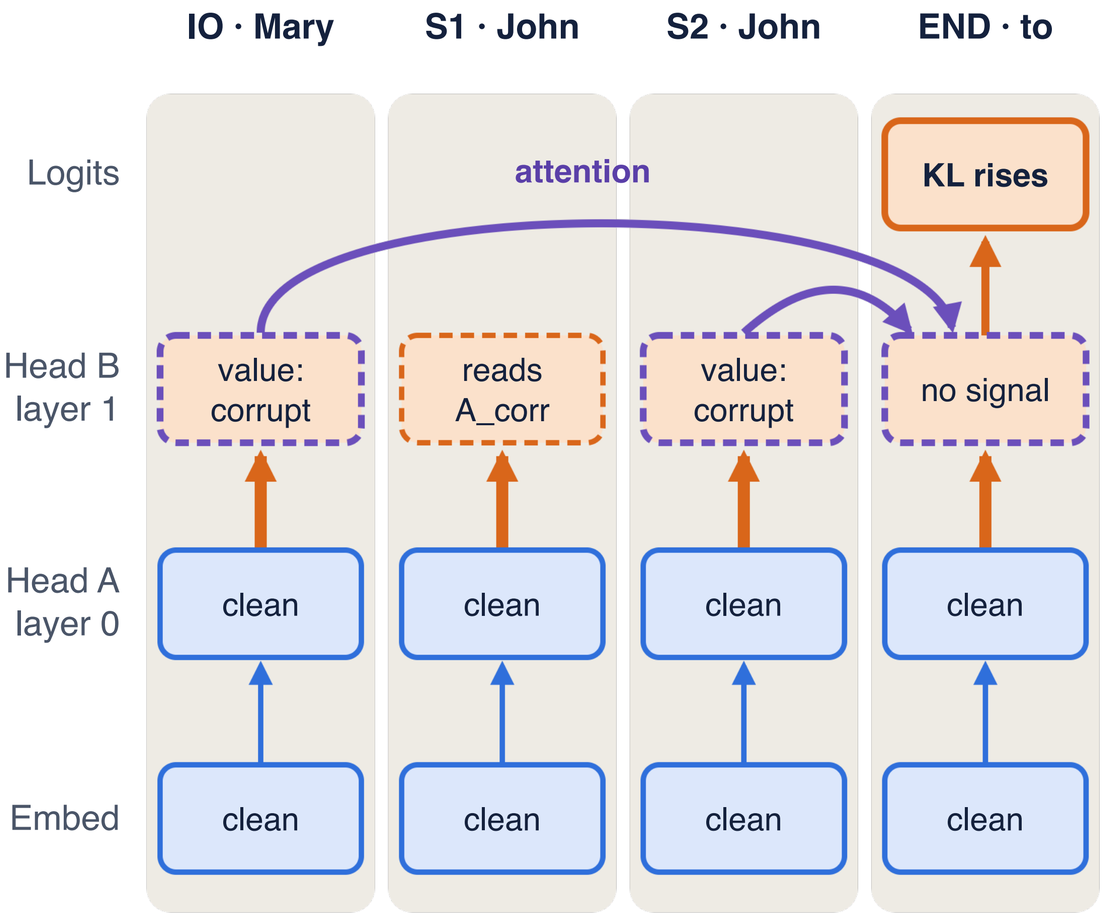


Circuit Discovery

Attention carries the damage to END

B at END still attends to IO and S2 as usual. But those values were built from A_corr, so what A wrote there (in IOI: “John is repeated” at S2) never arrives.

Read the logits at END. **ΔKL** = how much the KL to the full model rises because this edge was cut.



Circuit Discovery

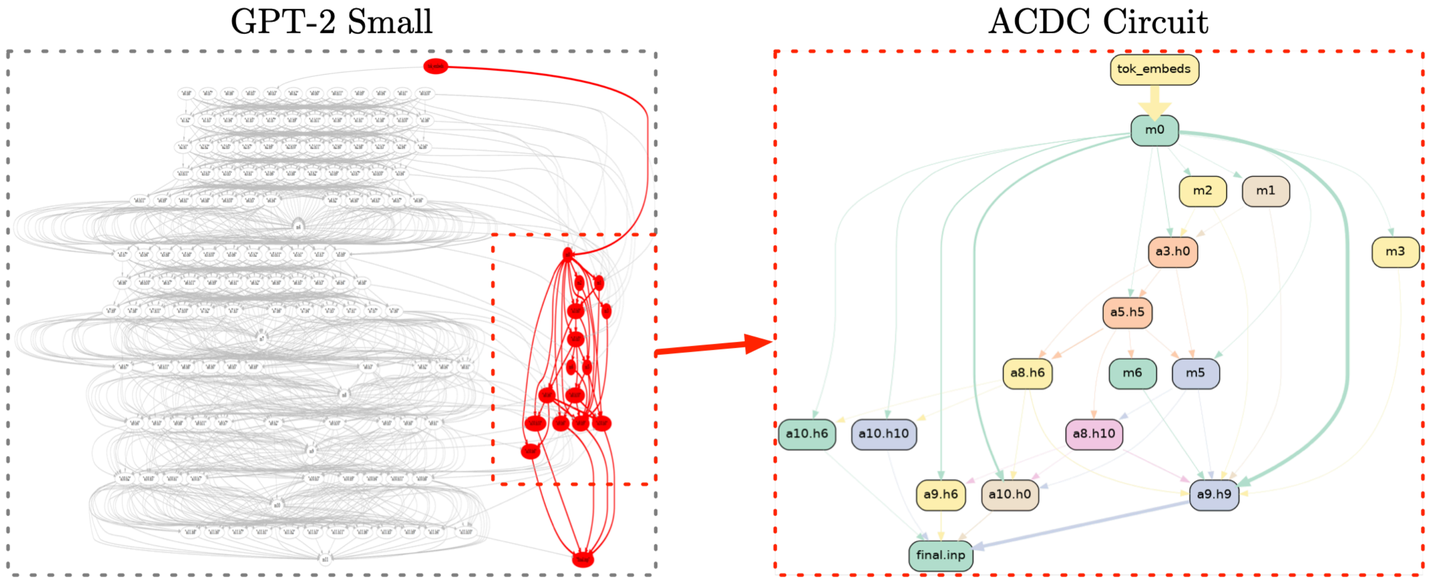


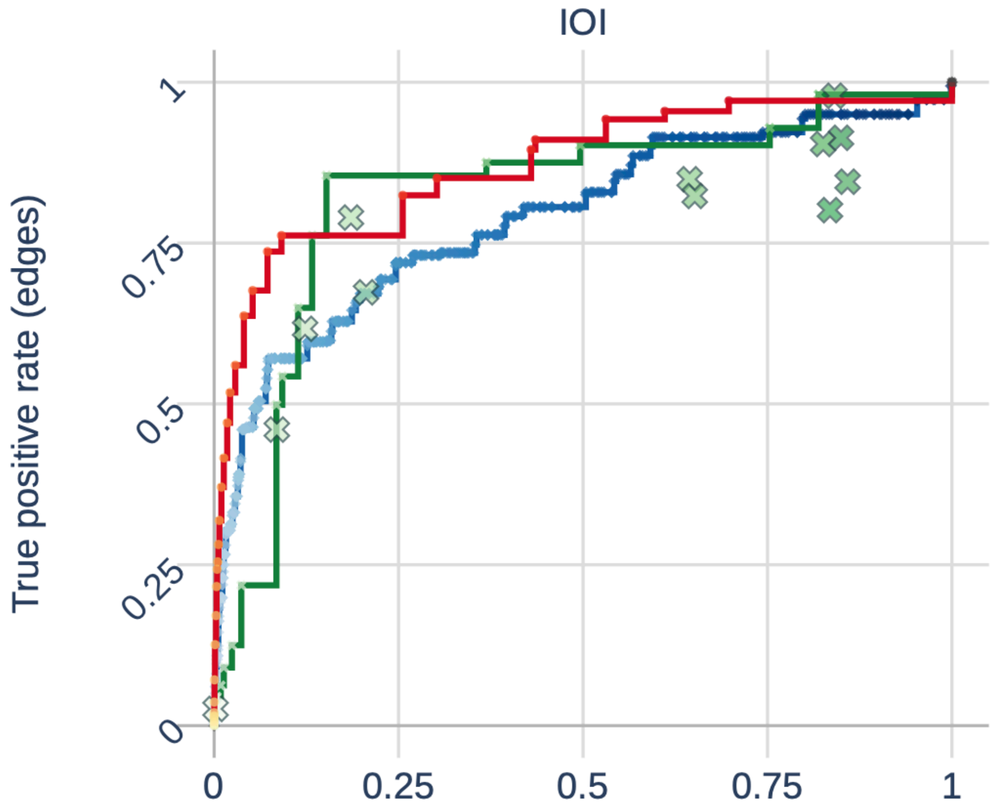
Figure 1: **Automatically discovering circuits with ACDC.** *Left:* a computational graph for GPT-2 Small, with a recovered circuit for the IOI task highlighted in red. Only edges between adjacent layers are shown. *Right:* the recovered circuit with labelled nodes. All heads recovered were identified as part of the IOI circuit by Wang et al. (2023). Edge thickness is proportional to importance.

Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

What does it give us? What is missing?

Circuit Discovery

- Evaluation: compare with manually engineered circuits.
- Treated as binary classification problems over edges.

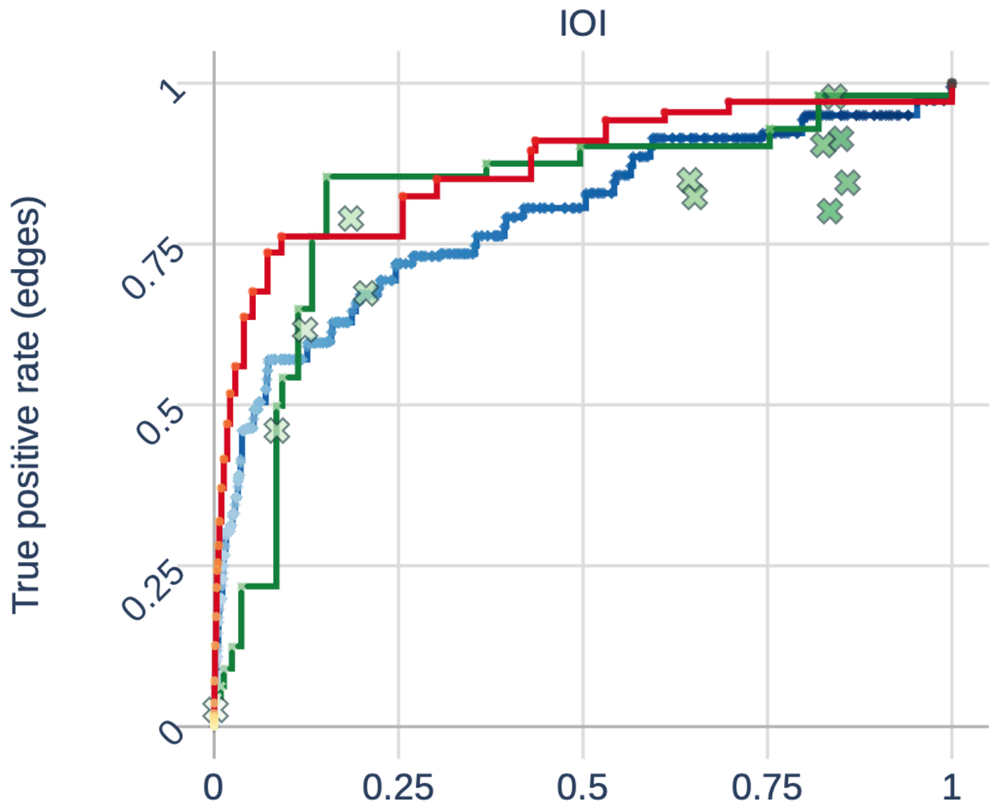


Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Circuit Discovery

- Evaluation: compare with manually engineered circuits.
- Treated as binary classification problems over edges.

Methodological issues?



Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Circuit Discovery

- Evaluation: compare with m
- Treated as binary classificati

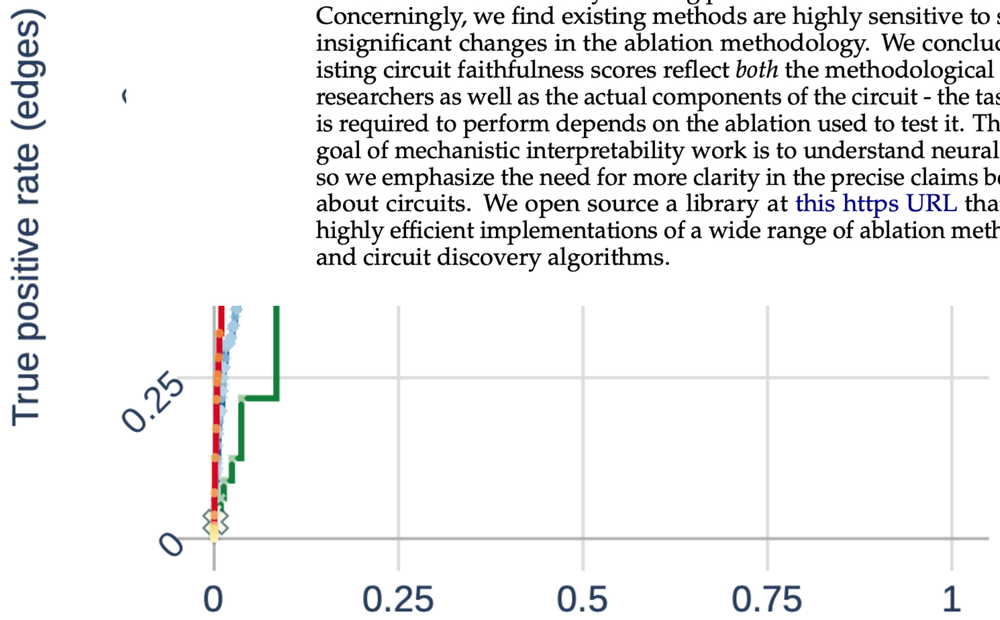
Methodological issues?

Transformer Circuit Faithfulness Metrics Are Not Robust

Joseph Miller* FAR AI
Bilal Chughtai Independent
William Saunders Independent

Abstract

Mechanistic interpretability work attempts to reverse engineer the learned algorithms present inside neural networks. One focus of this work has been to discover 'circuits' – subgraphs of the full model that explain behaviour on specific tasks. But how do we measure the performance of such circuits? Prior work has attempted to measure circuit 'faithfulness' – the degree to which the circuit replicates the performance of the full model. In this work, we survey many considerations for designing experiments that measure circuit faithfulness by ablating portions of the model's computation. Concerningly, we find existing methods are highly sensitive to seemingly insignificant changes in the ablation methodology. We conclude that existing circuit faithfulness scores reflect *both* the methodological choices of researchers as well as the actual components of the circuit - the task a circuit is required to perform depends on the ablation used to test it. The ultimate goal of mechanistic interpretability work is to understand neural networks, so we emphasize the need for more clarity in the precise claims being made about circuits. We open source a library at this https URL that includes highly efficient implementations of a wide range of ablation methodologies and circuit discovery algorithms.



Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Circuit Discovery

- Evaluation: compare with m
- Treated as binary classificati

Methodological issues?

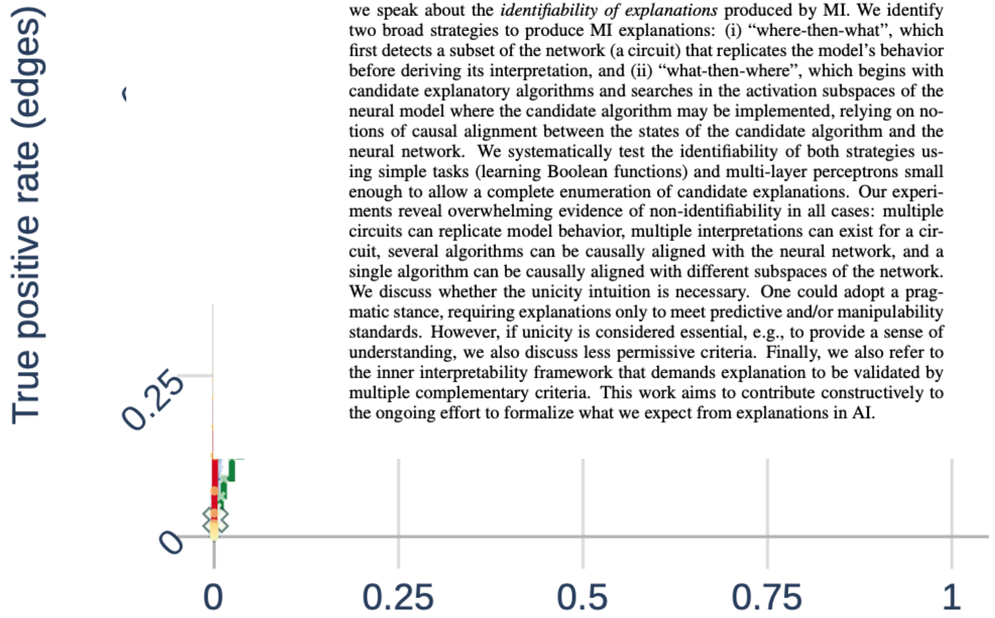
Published as a conference paper at ICLR 2025

EVERYTHING, EVERYWHERE, ALL AT ONCE: IS MECHANISTIC INTERPRETABILITY IDENTIFIABLE?

Maxime Méloux, François Portet, Silviu Maniu, Maxime Peyrard
Université Grenoble Alpes, CNRS, Grenoble INP, LIG, 38000 Grenoble, France
{melouxm,portetf,manius,peyrardm}@univ-grenoble-alpes.fr

ABSTRACT

As AI systems are increasingly deployed in high-stakes applications, ensuring their interpretability is essential. Mechanistic Interpretability (MI) aims to reverse-engineer neural networks by extracting human-understandable algorithms embedded within their structures to explain their behavior. This work systematically examines a fundamental question: for a fixed behavior to explain, and under the criteria that MI sets for itself, are we guaranteed a unique explanation? Drawing an analogy with the concept of *identifiability* in statistics, which ensures the uniqueness of parameters inferred from data under specific modeling assumptions, we speak about the *identifiability of explanations* produced by MI. We identify two broad strategies to produce MI explanations: (i) "where-then-what", which first detects a subset of the network (a circuit) that replicates the model's behavior before deriving its interpretation, and (ii) "what-then-where", which begins with candidate explanatory algorithms and searches in the activation subspaces of the neural model where the candidate algorithm may be implemented, relying on notions of causal alignment between the states of the candidate algorithm and the neural network. We systematically test the identifiability of both strategies using simple tasks (learning Boolean functions) and multi-layer perceptrons small enough to allow a complete enumeration of candidate explanations. Our experiments reveal overwhelming evidence of non-identifiability in all cases: multiple circuits can replicate model behavior, multiple interpretations can exist for a circuit, several algorithms can be causally aligned with the neural network, and a single algorithm can be causally aligned with different subspaces of the network. We discuss whether the unicity intuition is necessary. One could adopt a pragmatic stance, requiring explanations only to meet predictive and/or manipulability standards. However, if unicity is considered essential, e.g., to provide a sense of understanding, we also discuss less permissive criteria. Finally, we also refer to the inner interpretability framework that demands explanation to be validated by multiple complementary criteria. This work aims to contribute constructively to the ongoing effort to formalize what we expect from explanations in AI.



Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Circuit Discovery

- Evaluation: compare with m
- Treated as binary classificati

Methodological issues?

Published as a conference paper at ICLR 2025

EVERYTHING, EVERYWHERE, ALL AT ONCE:
IS MECHANISTIC INTERPRETABILITY IDENTIFIABLE?

Maxime Méloux, François Portet, Silviu Maniu, Maxime Peyrard
Université Grenoble Alpes, CNRS, Grenoble INP, LIG, 38000 Grenoble, France
{melouxm,portetf,manius,peyrardm}@univ-grenoble-alpes.fr

ABSTRACT

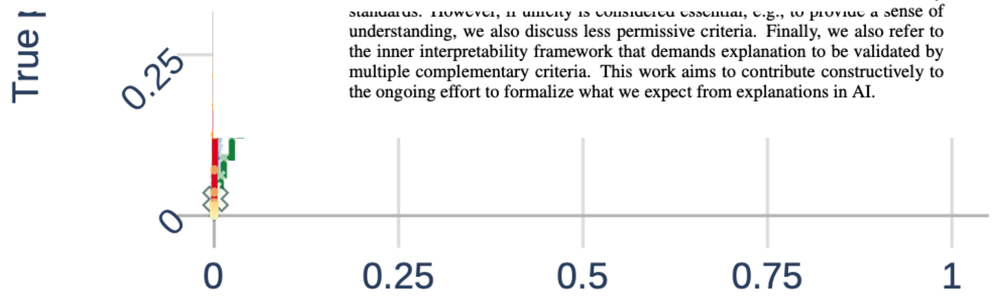
The Hydra Effect: Emergent Self-repair in Language Model Computations

Thomas McGrath[1], **Matthew Rahtz**[1], **János Kramár**[1], **Vladimir Mikulik**[1] **and Shane Legg**[1]
[1]Google DeepMind

We investigate the internal structure of language model computations using causal analysis and demonstrate two motifs: (1) a form of adaptive computation where ablations of one attention layer of a language model cause another layer to compensate (which we term the Hydra effect) and (2) a counterbalancing function of late MLP layers that act to downregulate the maximum-likelihood token. Our ablation studies demonstrate that language model layers are typically relatively loosely coupled (ablations to one layer only affect a small number of downstream layers). Surprisingly, these effects occur even in language models trained without any form of dropout. We analyse these effects in the context of factual recall and consider their implications for circuit-level attribution in language models.

ensur-
aims to
orithms
temati-
d under
Draw-
ures the
options,
dentify
, which
ehavior
ns with
s of the
on no-
and the
gies us-
s small
experi-
multiple
or a cir-
, and a
etwork.
a prag-
lability

standards. However, if unicity is considered essential, e.g., to provide a sense of understanding, we also discuss less permissive criteria. Finally, we also refer to the inner interpretability framework that demands explanation to be validated by multiple complementary criteria. This work aims to contribute constructively to the ongoing effort to formalize what we expect from explanations in AI.



Conmy, Arthur, et al. "Towards automated circuit discovery for mechanistic interpretability." *Advances in Neural Information Processing Systems* 36 (2023): 16318-16352.

Attribution patching

- Activation patching is expensive. Attribution patching is a fast approximation.
- Suppose we are interested in property L of the model (e.g. the ability to complete the capital of Paris, to predict the sentiment of a sentence, etc.)
- In activation patching for component E, we approximate

$$|L(x_{\text{clean}}|\ \text{do}(E = e_{\text{corr}})) - L(x_{\text{clean}})|$$

Syed, Aaquib, Can Rager, and Arthur Conmy. "Attribution patching outperforms automated circuit discovery." *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. 2024.

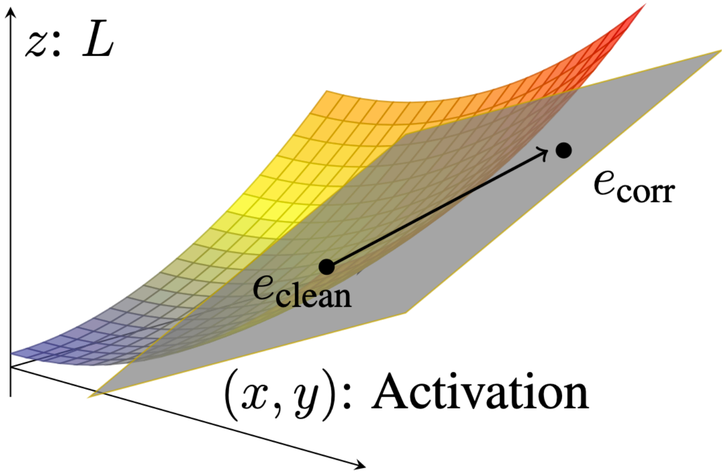
Attribution patching

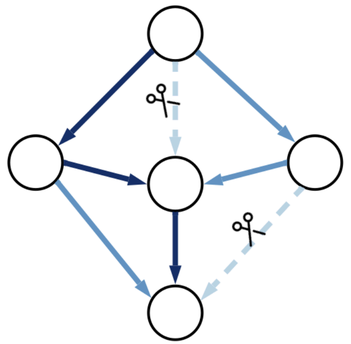
- In activation patching for component E, we approximate

$$|L(x_{\text{clean}}|\,\text{do}(E = e_{\text{corr}})) - L(x_{\text{clean}})|$$

- We replace this with the linear approximation

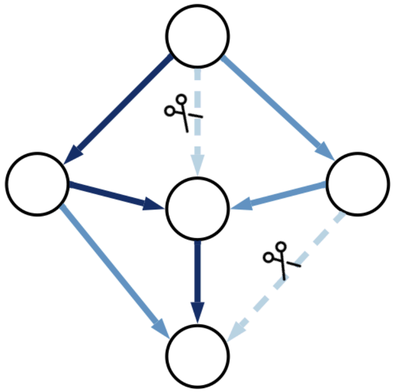
$$L(x_{\text{clean}}|\,\text{do}(E = e_{\text{corr}})) \approx L(x_{\text{clean}}) + \underbrace{(e_{\text{corr}} - e_{\text{clean}})^\top \frac{\partial}{\partial e_{\text{clean}}} L(x_{\text{clean}}|\,\text{do}(E = e_{\text{clean}}))}_{\text{Call this } \Delta_e L\text{, the } \textbf{attribution score}}.$$

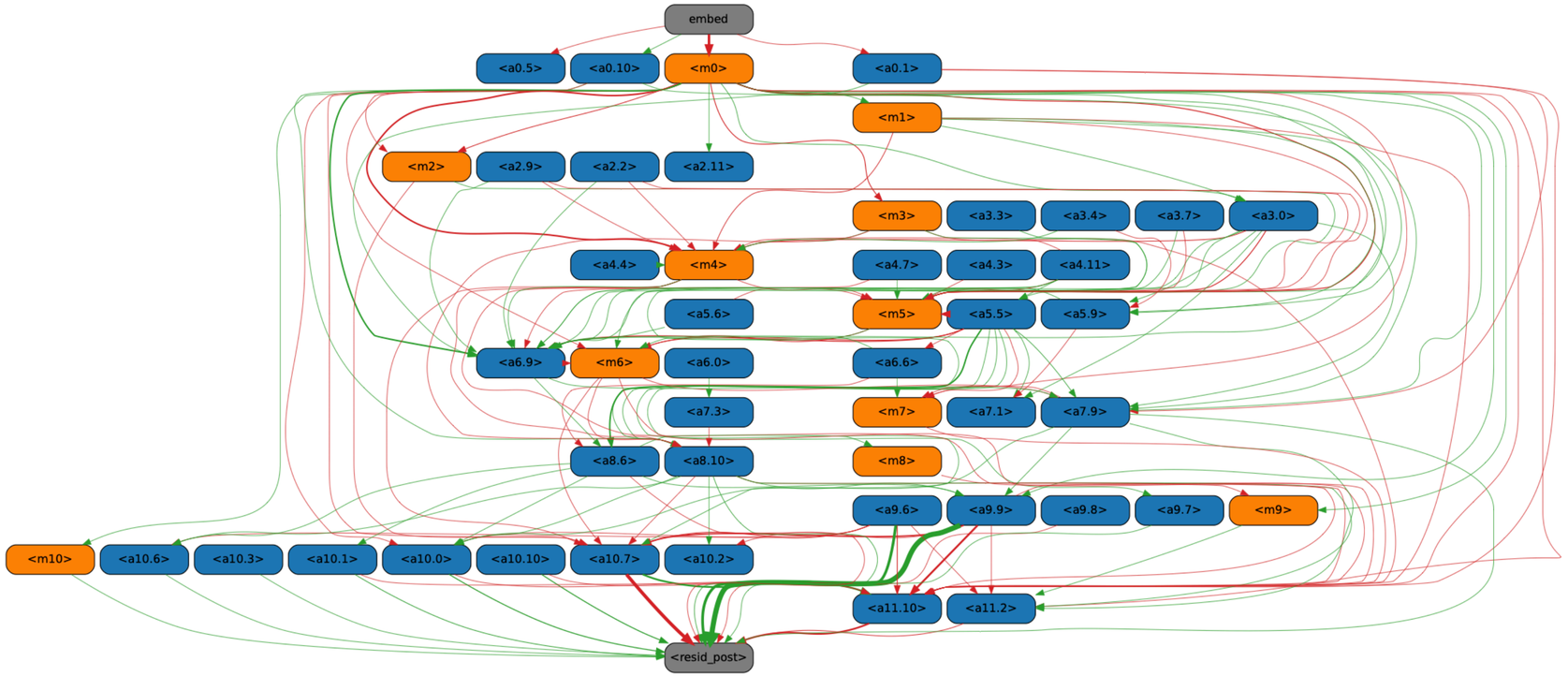




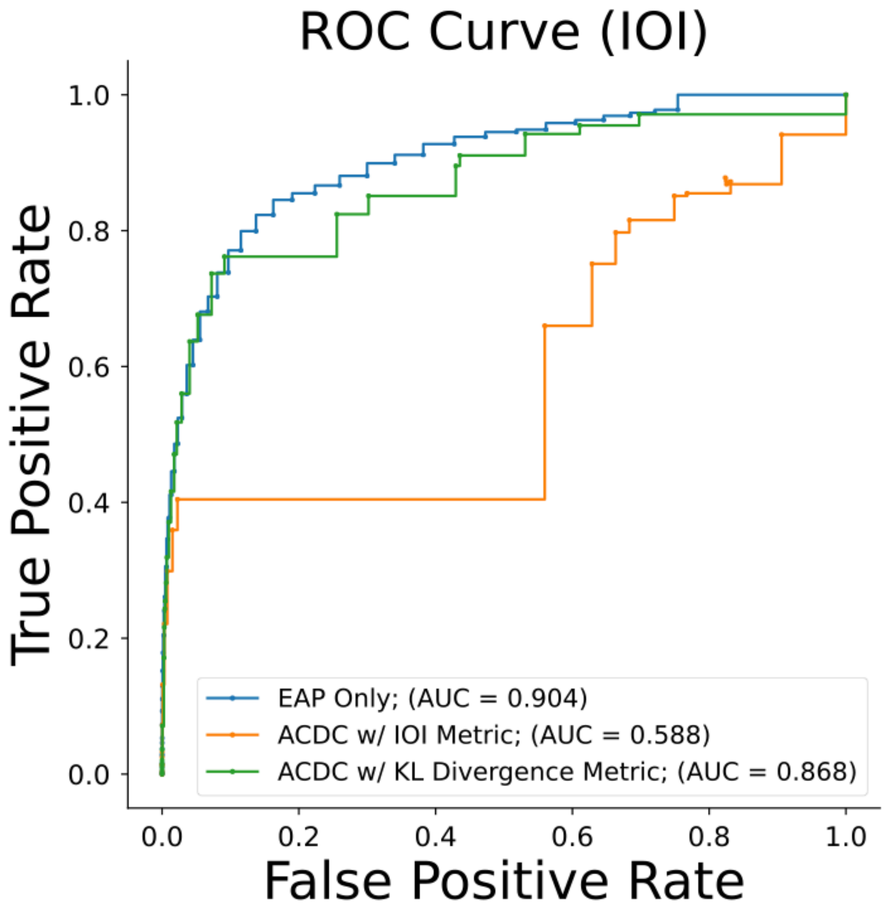
Attribution patching

We can use the insights from Section 3.2 to build an automated circuit discovery algorithm. This takes two steps: i) use Equation (2) to obtain absolute attribution scores for the importance of all edges in the computational graph and then ii) sort these scores and keep the top k edges in a circuit. We use **Edge Attribution Patching** (EAP) to refer to this algorithm. In the rest of the work we report results for all k values when we evaluate EAP (similar to HISP in [10]).





Attribution patching



(a) IOI task